

Anonymity of X-Wing and its Variants

Jiawei Bao and Jiaxin Pan 

University of Kassel, Kassel, Germany
{jiawei.bao,jiaxin.pan}@uni-kassel.de

Abstract. X-Wing (Barbosa et al., CiC Volume 1, Issue 1) is a hybrid key encapsulation mechanism (KEM) currently considered for standardization by IETF and deployed by major companies such as Google to ensure a secure transition to post-quantum cryptography. It combines a classical X25519 KEM with the post-quantum ML-KEM-768.

In this paper, we propose the first analysis of the anonymity of X-Wing. We are interested in tight and memory-tight reductions that offer stronger security guarantees. We first establish in the standard model that for any IND-CCA secure KEM, weak anonymity implies full anonymity, and our reduction is tight not only in success probability and time but also in memory consumption. We then prove in the random oracle model that X-Wing achieves weak anonymity if both X25519 and ML-KEM-768 are weakly anonymous. The former can even be proven without a hardness assumption.

Our proof on the weak anonymity of X-Wing does not preserve the memory-tightness of the underlying KEMs. To improve it, we propose a slight variant of X-Wing that preserves memory-tightness. Finally, we improve the existing IND-CCA proof of the original X-Wing by Barbosa et al. using our new memory-tight analysis.

Keywords. Anonymity, hybrid key encapsulation mechanism, post-quantum cryptography, memory tightness

1 Introduction

Key encapsulation mechanisms (KEMs) are arguably the most important primitive in public-key cryptography. They can be used to construct public-key encryption schemes [29], authenticated key exchange protocols (AKEs) [28,36], password-based key exchange protocols [7,37], and many more. Hence, their security is crucial.

HYBRID KEM. The development of large-scale quantum computers poses a fundamental threat to classical KEM or public-key cryptography in general. This is because schemes based on RSA and (elliptic-curve) Diffie–Hellman can be efficiently broken by Shor’s algorithm with a capable quantum computer. To address this risk, post-quantum cryptography (PQC) is constructing new algorithms [39,31] based on problems (such as lattice-based ones) that are assumed to be hard even to a quantum adversary.

However, relying solely on PQC schemes in critical systems is rather risky, since they are new and come with a lack of extensive cryptanalysis or mature

implementation. For instance, several previous implementations of Kyber [39] (which was recently used to derive the ML-KEM standard [34]) have been found to have timing vulnerabilities [12] and FPGA bitstream attacks [33]. Hence, the hybrid approach is recommended by many national security agencies, such as those from 18 EU member states¹ and the UK².

We are interested in hybrid KEMs in this paper. A hybrid KEM is essentially combining a classically deployed KEM with a post-quantum one in such a way that the combination is at least as secure as the deployed one, and the deployed KEM (e.g. the ElGamal KEM) has a long history of cryptanalysis and mature implementations. This design mitigates the uncertain security risks surrounding newly standardized PQC algorithms and ensures a smooth and secure transition toward quantum-safe cryptography in real-world applications such as TLS, SSH, and IKE. X-Wing [5] is among the well-known hybrid KEMs, which combines the ElGamal KEM on Curve25519 [30] and ML-KEM. It is very plausible that IETF will standardize X-Wing [16], and major companies like Google have already integrated X-Wing into their products³. This shows the pragmatic value of X-Wing during the post-quantum transitional period. Hence, it motivates us to better understand the security of X-Wing.

SECURITY OF X-WING. Standard security notion of a KEM is indistinguishability against chosen-ciphertext attacks (IND-CCA), where an adversary is asked to distinguish an honestly encapsulated key from random, given the ability to ask for decryption on adaptively chosen ciphertexts. Beyond IND-CCA, anonymity (or key privacy) is another important security guarantee for KEM. In particular, it is required when it is used to construct password-based key exchange protocols [7,37]. Other well-known applications of anonymous KEM (or anonymous PKE) schemes include the deployed Zcash system [11], anonymous credential systems [14], auction protocols [38], and anonymous key exchange protocols [35].

We are interested in the anonymity of X-Wing, since it will lead to new hybrid constructions of the aforementioned applications and safeguard the post-quantum transition. In an anonymous KEM, we require that the ciphertexts and the encapsulated keys leak no information about the corresponding public key. A weaker form of anonymity requires only the ciphertexts are (computationally) independent of the public key. As in the IND security, we allow an anonymity adversary to perform CCA attacks. We denote the (weak) anonymity against CCA as ANO-CCA and wANO-CCA respectively. Our goal is to achieve ANO-CCA.

¹ <https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Crypto/PQC-joint-statement.pdf>

² https://www.ncsc.gov.uk/whitepaper/next-steps-preparing-for-post-quantum-cryptography#section_5

³ <https://bughunters.google.com/blog/5266882047639552/why-hybrid-deployments-are-key-to-secure-pqc-migration>

1.1 Our contributions

This work has two major contributions, formally proving the anonymity of X-Wing in the random oracle model and proposing a slight variant of X-Wing whose security proof preserves not only (probability-time) tightness of the underlying KEMs, but also their memory-tightness [4]. Our results show that X-Wing (or our new variant) can serve as a drop-in replacement for the previously mentioned anonymous-KEM-based protocol, enabling a hybrid implementation that ensures a secure PQC transition. We detail our contributions here.

INTERLUDE: TIGHT SECURITY AND MEMORY TIGHTNESS. Reductions with tightness on success probability, running time, and memory are preferable, since they provide stronger security guarantees. A classical understanding of tight security [10,8,9] is that a reduction $\mathcal{R}$ between a cryptographic problem P and a scheme S preserves the *success probability* and *running time* of the adversary $\mathcal{A}$ against S . Concretely, imagine that $\mathcal{A}$ breaks S with probability $\varepsilon_{\mathcal{A}}$ and running time $t_{\mathcal{A}}$ and using $\mathcal{A}$, $\mathcal{R}$ can break P with probability $\varepsilon_{\mathcal{R}}$ and running time $t_{\mathcal{R}}$. We say the reduction $\mathcal{R}$ is tight if $t_{\mathcal{A}} \approx t_{\mathcal{R}}$ and $\varepsilon_{\mathcal{A}} \leq L \cdot \varepsilon_{\mathcal{R}}$ with a small constant L (e.g., $L = 1$ or 2). With a tight reduction, a practitioner does not need to adjust the parameters of the underlying instance of P , giving us more efficient implementation.

Recently, it has been discovered that some cryptographic problems are memory-sensitive, namely, with more memory these problems become easier to solve. The lattice problem underlying the ML-KEM-768 is one of such problems [15,2,26,6,25]. Hence, a reduction that consumes large memory does not make much sense, since in this case the underlying memory-sensitive problem may not be hard. In our analysis of X-Wing we also focus on the memory consumption of our reductions and in many cases we can construct memory-tight reductions. The study of memory-tight reductions was initiated by Auerbach et al. [4], and a reduction is memory-tight if $m_{\mathcal{R}} \approx m_{\mathcal{A}}$ where $m_{\mathcal{A}}$ and $m_{\mathcal{R}}$ are the amount of memory used by $\mathcal{A}$ and $\mathcal{R}$. We refer [4] for more discussion on the motivation of memory-tightness.

FORMAL ANALYSIS OF X-WING. We formally prove the anonymity of X-Wing. We first establish a general lemma that, for any IND-CCA secure KEM, its wANO-CCA implies ANO-CCA. This lemma is proven in the standard model. After that, we consider the anonymity of X-Wing. More precisely, we prove that

- (1) If the ML-KEM-768 is IND-CCA secure, then the ANO-CCA security of X-Wing is implied by the wANO-CCA and WCPR-CCA security of ML-KEM-768 and wANO-PCA security of X25519. Here ‘WCPR’ stands for weak chosen-preimage resistance, which states that if we use a different secret key to decapsulate a challenge ciphertext then we get a pseudorandom outcome. And ‘PCA’ stands for plaintext-checking attacks, where adversaries can submit plaintext-ciphertext pairs to check if they correspond to each other. The security proof here is *memory-tight*.
- (2) If the ML-KEM-768 is not IND-CCA secure, then the wANO-CCA security of X-Wing is implied by the wANO-PCA security of both X25519 and ML-

KEM-768. By the general lemma, this yields ANO-CCA security of X-Wing. In fact, this second implication achieves a stronger wANO-CCA security of X-Wing, namely, an anonymity adversary is allowed to decapsulate a challenge ciphertext, but it is not memory-tight.

Both implications are proven in the random oracle model. Our proofs require both X25519 and ML-KEM-768 to be weakly anonymous, otherwise the anonymity of X-Wing will be trivially broken. Previously, a similar observation was made in [24], but we relax it to weak anonymity.

MEMORY-TIGHT ANONYMOUS KEM. We now turn to the discussion of the (memory) tightness of our reductions. The generic “wANO-CCA-to-ANO-CCA” implication is tight in terms of success probability, time and memory. However, the existing IND-CCA security proof of X-Wing is not memory-tight [5]. In Theorem 7, we prove the IND-CCA security of X-Wing with a memory-tight reduction, which improves the work of Barbosa [5]. But the reduction in the above Item (2) is not memory-tight.

Motivated by this, we make one modification to X-Wing, namely, we include ciphertexts and keys from both X25519 and ML-KEM-768 into the hash function that derives the final encapsulation key. By doing so, we can simulate the random oracle without keeping its history, and thus it yields a memory-tight proof.

OPEN PROBLEMS. We leave proving the security of X-Wing in the quantum random oracle model (QROM) as an interesting open problem. This seems technically challenging, since our anonymity proof must abort if the adversary queries the random oracle at the challenge key, which the proof does not know. Even in the classical random oracle model, it is already non-trivial. In the QROM, one cannot apply standard reprogramming techniques, such as the One-Way-to-Hiding lemma [40], to reprogram the challenge point without knowledge of its position.

1.2 Related work

Very recently, Günther et al. [24] proposed a new KEM combiner that can achieve strong pseudorandomness under chosen-ciphertext attacks (SPR-CCA) (as defined in [41]), which implies ANO-CCA. Interestingly, the anonymity of their combiner does not require both their underlying KEMs to be anonymous, but rather the outer one to have statistically uniform ciphertexts (which can be satisfied by the ElGamal KEM). This improves over X-Wing. Moreover, we note that the underlying KEMs have to be SPR-CCA secure in [24], which is stronger than the weak anonymity required in our proofs, since the SPR-CCA security additionally allows adversaries to possess the encapsulated key to the challenge ciphertext. To achieve the combination, their construction relies on less standard symmetric primitives, including a length-preserving symmetric encryption scheme, a split-key PRF, and a PRG.

Grubbs et al. [23] initiated the analysis of post-quantum KEM (excluding hybrid KEMs). They proved the anonymity of Fujisaki-Okamoto (FO) transformations and its variants [18, 19, 20, 27]. They mainly focused on the case of implicit

rejection and show the ANO-CCA security of $\text{FO}^\perp$ in the QROM from the weak anonymity, the one-wayness and the strong collision freeness (SCFR-CPA) of the underlying PKE. Compared with their work, our “wANO-to-ANO” lemma (cf. Lemma 1) is more general. In addition, they analyzed the anonymity of three specific NIST PQC Candidates, including Classic McEliece (CM) [1], proto-Saber [17] and FrodoKEM [32].

2 Preliminaries

For a (finite) set X , we write $x \xleftarrow{\$} X$ for sampling an element x uniformly at random from X . All sets are initialized to the empty set unless stated otherwise. We assume that all algorithms and adversaries in this paper are probabilistic; otherwise, we will state it. For an algorithm, we write $y \leftarrow \mathcal{A}(x_1, \dots, x_n)$ when $\mathcal{A}$ outputs y on input $(x_1, \dots, x_n)$. We may write the randomness r explicitly as $y := \mathcal{A}(x_1, \dots, x_n; r)$. If an algorithm $\mathcal{A}$ has access to an oracle $\mathcal{O}$, we write $\mathcal{A}^\mathcal{O}$. We follow the notations of reduced complexity as in [21] to analyze the time and memory complexity of algorithms, namely, if $\mathcal{A}^\mathcal{O}$ is an adversary that calls another adversary $\hat{\mathcal{A}}$ and provides $\hat{\mathcal{A}}$ with oracle access to $\mathcal{O}$ and H (which is either a pseudo-random function or a random function), then $\mathbf{Time}^*(\mathcal{A}) := \mathbf{Time}(\hat{\mathcal{A}})$ and $\mathbf{Mem}^*(\mathcal{A}) := \mathbf{Mem}(\hat{\mathcal{A}})$. Essentially, the reduced complexity is reasonable, since it excludes the cost for simulating a hash function. Also, the oracle $\mathcal{O}$ is external to $\mathcal{A}$, and thus its complexity does not count towards $\mathcal{A}$. We use $\mathcal{O}$ to denote asymptotic upper bounds, indicating growth up to a constant factor.

2.1 Key Encapsulation Mechanism

Let par be system parameters. A key encapsulation mechanism KEM consists of three algorithms (KeyGen, Encap, Decap) defined as follows:

- The key generation algorithm **KeyGen** is an algorithm that on input par outputs a public and secret key pair (pk, sk) .
- The encapsulation algorithm **Encap** is an algorithm that on input of a public key pk outputs a pair (k, c) , where k is the encapsulated key and c is the encapsulation ciphertext.
- The decapsulation algorithm **Decap** is a deterministic algorithm that on input of a secret key sk and a ciphertext c outputs a key k or a special symbol $\perp \notin \mathcal{M}$ indicating that c is invalid.

Definition 1 (KEM Correctness). *Let $\text{KEM} = (\text{KeyGen}, \text{Encap}, \text{Decap})$ be a key encapsulation mechanism. We say that KEM is δ -correct if*

$$\Pr \left[\text{Decap}(\text{sk}, c) \neq k \mid \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{KeyGen} \\ (k, c) \leftarrow \text{Encap}(\text{pk}) \end{array} \right] \leq \delta.$$

Our security analysis also requires the ciphertext second preimage resistance [5] (C2PRI) of a KEM that is defined as follows.

Definition 2 (C2PRI). *The C2PRI advantage of an adversary $\mathcal{A}$ against $\text{KEM} = (\text{KeyGen}, \text{Encap}, \text{Decap})$ is defined as*

$$\text{Adv}_{\text{KEM}}^{\text{C2PRI}}(\mathcal{A}) = \Pr \left[\text{Decap}(\text{sk}, c) = k^* \wedge c \neq c^* \mid \begin{array}{l} (pk, sk) \leftarrow \text{KeyGen}(\text{par}) \\ (k^*, c^*) \leftarrow \text{Encap}(pk) \\ c \leftarrow \mathcal{A}(pk, sk, k^*, c^*) \end{array} \right].$$

Definition 3 (Collision of the public keys). *The collision advantage of the public keys of $\text{KEM} = (\text{KeyGen}, \text{Encap}, \text{Decap})$ is defined as*

$$\text{Adv}_{\text{KEM}}^{\text{COLL}} = \Pr \left[pk_0 = pk_1 \mid \begin{array}{l} (pk_0, sk_0) \leftarrow \text{KeyGen}(\text{par}) \\ (pk_1, sk_1) \leftarrow \text{KeyGen}(\text{par}) \end{array} \right].$$

We define seven different security notions in Definition 4 that are needed here: ANO and wANO stand for (weak) anonymity respectively, WCPR for weak chosen-preimage resistance, IND for indistinguishability, OW for one-wayness, PCA for plaintext-checking attacks, and CCA for chosen-ciphertext attacks. CCA^* is a stronger form of CCA, where an adversary $\mathcal{A}$ can query decapsulation on the challenge ciphertext c^* . WCPR-CCA, wANO- CCA^* , and wANO-PCA are new notions defined in this paper as intermediate notions for our (later) security analysis. The remaining ones are taken from [23].

Definition 4 (Security notions for KEM). *For $\text{KEM} = (\text{KeyGen}, \text{Encap}, \text{Decap})$, let $\mathcal{A}$ be an adversary against KEM; we define a set of security notions using the games in Figure 1. For $X\text{-}Y \in \{\text{ANO-CCA}, \text{wANO-PCA}, \text{wANO-CCA}, \text{wANO-CCA}^*, \text{WCPR-CCA}, \text{IND-CCA}\}$, the advantage of $\mathcal{A}$ is defined as*

$$\text{Adv}_{\text{KEM}}^{X\text{-}Y}(\mathcal{A}) = |\Pr[X\text{-}Y_{\text{KEM}}^{\mathcal{A}} \Rightarrow 1] - 1/2|. \quad (1)$$

For $X\text{-}Y \in \{\text{OW-PCA}\}$, we define the advantage of $\mathcal{A}$ as

$$\text{Adv}_{\text{KEM}}^{X\text{-}Y}(\mathcal{A}) = \Pr[X\text{-}Y_{\text{KEM}}^{\mathcal{A}} \Rightarrow 1]. \quad (2)$$

We say KEM is $X\text{-}Y$ secure if for any efficient adversary $\mathcal{A}$, $\text{Adv}_{\text{KEM}}^{X\text{-}Y}(\mathcal{A})$ is negligible.

2.2 Public-key Encryption

A public-key encryption scheme PKE consists of three algorithms (Gen, Enc, Dec) defined as follows:

- The key generation algorithm Gen is an algorithm that on input par outputs a public and secret key pair (pk, sk) .
- The encryption algorithm Enc is an algorithm that on input of a public key pk and a message m from the message space $\mathcal{M}$ outputs a ciphertext c .
- The decryption algorithm Dec is a deterministic algorithm that on input of a secret key sk and a ciphertext c outputs a message m or a special symbol $\perp \notin \mathcal{M}$ indicating that c is invalid.

ANO-CCA_{KEM}^A 01 $(pk_0, sk_0) \leftarrow \text{KeyGen}(\text{par}_0)$ 02 $(pk_1, sk_1) \leftarrow \text{KeyGen}(\text{par}_1)$ 03 $b \xleftarrow{\$} \{0, 1\}$ 04 $(c^*, k^*) \leftarrow \text{Encap}(pk_b)$ 05 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\cdot, \cdot)}(pk_0, pk_1, (c^*, k^*))$ 06 $b' \leftarrow \mathcal{A}^{\text{PCO}(\cdot, \cdot)}(pk_0, pk_1, c^*)$ 07 return $\llbracket b = b' \rrbracket$	wANO-PCA_{KEM}^A 01 $(pk_0, sk_0) \leftarrow \text{KeyGen}(\text{par}_0)$ 02 $(pk_1, sk_1) \leftarrow \text{KeyGen}(\text{par}_1)$ 03 $b \xleftarrow{\$} \{0, 1\}$ 04 $(c^*, k^*) \leftarrow \text{Encap}(pk_b)$ 05 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\cdot, \cdot)}(pk_0, pk_1, (c^*, k^*))$ 06 $b' \leftarrow \mathcal{A}^{\text{PCO}(\cdot, \cdot)}(pk_0, pk_1, c^*)$ 07 return $\llbracket b = b' \rrbracket$	wANO-CCA_{KEM}^A 20 $(pk_0, sk_0) \leftarrow \text{KeyGen}(\text{par}_0)$ 21 $(pk_1, sk_1) \leftarrow \text{KeyGen}(\text{par}_1)$ 22 $b \xleftarrow{\$} \{0, 1\}$ 23 $(c^*, k^*) \leftarrow \text{Encap}(pk_b)$ 24 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\cdot, \cdot)}(pk_0, pk_1, c^*)$ 25 $b' \leftarrow \mathcal{A}^{\text{DECAPO}^*(\cdot, \cdot)}(pk_0, pk_1, c^*)$ 26 return $\llbracket b = b' \rrbracket$	wANO-CCA_{KEM}^{A*} 20 $(pk_0, sk_0) \leftarrow \text{KeyGen}(\text{par}_0)$ 21 $(pk_1, sk_1) \leftarrow \text{KeyGen}(\text{par}_1)$ 22 $b \xleftarrow{\$} \{0, 1\}$ 23 $(c^*, k^*) \leftarrow \text{Encap}(pk_b)$ 24 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\cdot, \cdot)}(pk_0, pk_1, c^*)$ 25 $b' \leftarrow \mathcal{A}^{\text{DECAPO}^*(\cdot, \cdot)}(pk_0, pk_1, c^*)$ 26 return $\llbracket b = b' \rrbracket$	
WCPR-CCA_{KEM}^A 08 $(pk_0, sk_0) \leftarrow \text{KeyGen}(\text{par}_0)$ 09 $(pk_1, sk_1) \leftarrow \text{KeyGen}(\text{par}_1)$ 10 $b \xleftarrow{\$} \{0, 1\}$ 11 $(c^*, k) \leftarrow \text{Encap}(pk_b)$ 12 $k_0 := \text{Decap}(sk_{1-b}, c^*)$ 13 $k_1 \xleftarrow{\$} \mathcal{K}$ 14 $b' \xleftarrow{\$} \{0, 1\}$ 15 $\hat{b} \leftarrow \mathcal{A}^{\text{DECAPO}(\cdot, \cdot)}(pk_0, pk_1, b, c^*, k_{b'})$ 16 return $\llbracket \hat{b} = b' \rrbracket$	IND-CCA_{KEM}^A 27 $(pk, sk) \leftarrow \text{KeyGen}(\text{par})$ 28 $(c^*, k_0) \leftarrow \text{Encap}(pk)$ 29 $k_1 \xleftarrow{\$} \mathcal{K}$ 30 $b \xleftarrow{\$} \{0, 1\}$ 31 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\cdot, \cdot)}(pk, c^*, k_b)$ 32 $k \leftarrow \mathcal{A}^{\text{PCO}(\cdot, \cdot)}(pk, c^*)$ 33 return $\llbracket b = b' \rrbracket$ 34 return $\llbracket k_0 = k \rrbracket$	OW-PCA_{KEM}^A 27 $(pk, sk) \leftarrow \text{KeyGen}(\text{par})$ 28 $(c^*, k_0) \leftarrow \text{Encap}(pk)$ 29 $k_1 \xleftarrow{\$} \mathcal{K}$ 30 $b \xleftarrow{\$} \{0, 1\}$ 31 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\cdot, \cdot)}(pk, c^*, k_b)$ 32 $k \leftarrow \mathcal{A}^{\text{PCO}(\cdot, \cdot)}(pk, c^*)$ 33 return $\llbracket b = b' \rrbracket$ 34 return $\llbracket k_0 = k \rrbracket$	Oracle DECAPO[*](pk, c) 17 if $c = c^*$: return $\perp$ 18 $k := \text{Decap}(sk, c)$ 19 return k	Oracle PCO(pk, k, c) 35 if $\text{Decap}(sk, c) = k$ 36 return 1 37 else 38 return 0

Fig. 1. Security games for KEM. Boxed codes are only executed in the corresponding games. Without loss of generality, here we assume that $\mathcal{A}$ is allowed to query the oracles only with the public key pk in games IND-CCA and OW-PCA, and with pk_0 or pk_1 in the other games. The sk in the oracles denotes the corresponding secret key generated together with the input public key pk .

Definition 5 (PKE Correctness [27]). Let $\text{PKE} = (\text{Gen}, \text{Enc}, \text{Dec})$ be a public-key encryption scheme. We say that PKE is δ -correct if

$$\mathbb{E} \left[\max_{m \in \mathcal{M}} \Pr[\text{Dec}(sk, c) \neq m \mid c \leftarrow \text{Enc}(pk, m)] \right] \leq \delta,$$

where the expectation is taken over the distribution over all the key pairs $(pk, sk) \leftarrow \text{Gen}(\text{par})$.

Definition 6 (PKE γ -spreadness [19]). Let $\text{PKE} = (\text{Gen}, \text{Enc}, \text{Dec})$ be a public-key encryption scheme. We say that PKE is γ -spread if for any key pair (pk, sk) , message $m \in \mathcal{M}$, and ciphertext $c \in \mathcal{C}$,

$$\Pr[c = \text{Enc}(pk, m; r) \mid r \xleftarrow{\$} \mathcal{R}] \leq 2^{-\gamma}.$$

We define seven different security notions in Definition 7 that are needed here: ANO and wANO stand for (weak) anonymity, WCFR weak collision freeness, OW one-wayness, PCA plaintext-checking attacks, CPA chosen-plaintext attacks, and CCA chosen-ciphertext attacks. All these notions are taken from [23].

Definition 7 (Security notions for PKE). For $\text{PKE} = (\text{Gen}, \text{Enc}, \text{Dec})$, let $\mathcal{A}$ be an adversary against PKE; we define a set of security notions using the games in Figure 2. For $\text{X-Y} \in \{\text{ANO-CCA}, \text{ANO-CPA}, \text{wANO-CCA}, \text{wANO-CPA}\}$, the advantage of $\mathcal{A}$ is defined as

$$\text{Adv}_{\text{PKE}}^{\text{X-Y}}(\mathcal{A}) = |\Pr[\text{X-Y}_{\text{PKE}}^{\mathcal{A}} \Rightarrow 1] - 1/2|. \quad (3)$$

For $\text{X-Y} \in \{\text{WCFR-CCA}, \text{WCFR-CPA}, \text{OW-CPA}\}$, we define the advantage of $\mathcal{A}$ as

$$\text{Adv}_{\text{PKE}}^{\text{X-Y}}(\mathcal{A}) = \Pr[\text{X-Y}_{\text{PKE}}^{\mathcal{A}} \Rightarrow 1]. \quad (4)$$

We say PKE is X-Y secure if for any efficient adversary $\mathcal{A}$, $\text{Adv}_{\text{PKE}}^{\text{X-Y}}(\mathcal{A})$ is negligible.

ANO-CCA_{PKE}^A ANO-CPA_{PKE}^A 01 $(pk_0, sk_0) \leftarrow \text{Gen}(\text{par}_0)$ 02 $(pk_1, sk_1) \leftarrow \text{Gen}(\text{par}_1)$ 03 $b \xleftarrow{\\$} \{0, 1\}$ 04 $(m, st) \leftarrow \mathcal{A}^{\text{DECO}(\cdot, \cdot)}(pk_0, pk_1)$ 05 $c^* \leftarrow \text{Enc}(pk_b, m)$ 06 $b' \leftarrow \mathcal{A}^{\text{DECO}_{\mathcal{G}^*}(\cdot, \cdot)}(c^*, st)$ 07 $b' \leftarrow \mathcal{A}(c^*, st)$ 08 return $\llbracket b' = b \rrbracket$	wANO-CCA_{PKE}^A wANO-CPA_{PKE}^A 16 $(pk_0, sk_0) \leftarrow \text{Gen}(\text{par}_0)$ 17 $(pk_1, sk_1) \leftarrow \text{Gen}(\text{par}_1)$ 18 $b \xleftarrow{\\$} \{0, 1\}$ 19 $m^* \xleftarrow{\\$} \mathcal{M}$ 20 $c^* \leftarrow \text{Enc}(pk_b, m^*)$ 21 $b' \leftarrow \mathcal{A}^{\text{DECO}_{\mathcal{G}^*}(\cdot, \cdot)}(pk_0, pk_1, c^*)$ 22 $b' \leftarrow \mathcal{A}(pk_0, pk_1, c^*)$ 23 return $\llbracket b' = b \rrbracket$
WCFR-CCA_{PKE}^A WCFR-CPA_{PKE}^A 09 $(pk_0, sk_0) \leftarrow \text{Gen}(\text{par}_0)$ 10 $(pk_1, sk_1) \leftarrow \text{Gen}(\text{par}_1)$ 11 $(m, b) \leftarrow \mathcal{A}^{\text{DECO}(\cdot, \cdot)}(pk_0, pk_1)$ 12 $(m, b) \leftarrow \mathcal{A}(pk_0, pk_1)$ 13 $c \leftarrow \text{Enc}(pk_b, m)$ 14 $m' := \text{Dec}(sk_{1-b}, c)$ 15 return $\llbracket m' = m \wedge m \neq \perp \rrbracket$	OW-CPA_{PKE}^A Oracle $\text{DECO}(pk_b, c)$ 24 $(pk, sk) \leftarrow \text{Gen}(\text{par})$ 25 $m^* \xleftarrow{\\$} \mathcal{M}$ 26 $c^* \leftarrow \text{Enc}(pk, m^*)$ 27 $m \leftarrow \mathcal{A}(pk, c^*)$ 28 return $\llbracket m = m^* \rrbracket$ 29 if $c = c^* : \text{return } \perp$ 30 $m := \text{DECO}(sk_b, c)$ 31 return m

Fig. 2. Security games for PKE. Boxed codes are only executed in the corresponding games. Without loss of generality, here we assume that $\mathcal{A}$ can only query the oracles with the public keys pk_0 or pk_1 .

3 From Weak to Standard Anonymity

The standard anonymity requirement is ANO-CCA. In some intermediate steps of our proofs in this paper, we often show that a KEM is weakly anonymous. The following Lemma 1 shows that if a KEM is IND-CCA secure, then its wANO-CCA security implies the ANO-CCA security. Hence, it is sufficient to prove the weaker anonymity (wANO-CCA) of a KEM. Our proof of Lemma 1 preserves the tightness of a KEM in terms of advantage, time, and memory.

Lemma 1 (wANO-CCA+IND-CCA $\Rightarrow$ ANO-CCA). *Let $\text{KEM} = (\text{KeyGen}, \text{Encap}, \text{Decap})$ be a key encapsulation mechanism. Let $\mathcal{A}$ be an adversary against the ANO-CCA security of KEM, making at most q_D queries to the decapsulation oracles. Then there exists a wANO-CCA adversary $\mathcal{A}_1$ and an IND-CCA adversary $\mathcal{A}_2$ against KEM such that*

$$\text{Adv}_{\text{KEM}}^{\text{ANO-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{KEM}}^{\text{wANO-CCA}}(\mathcal{A}_2) + \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}_1), \quad (5)$$

and

$$\begin{aligned} \text{Time}^*(\mathcal{A}_1) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{A}_1) &\approx \text{Mem}(\mathcal{A}), \\ \text{Time}^*(\mathcal{A}_2) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{A}_2) &\approx \text{Mem}(\mathcal{A}). \end{aligned}$$

PROOF. Let $\mathcal{A}$ be an adversary against the ANO-CCA security of KEM. Consider the sequence of games $\mathbf{G}_0$ - $\mathbf{G}_1$ shown in Figure 3.

Game $\mathbf{G}_0$ - $\mathbf{G}_1$	
01 $(\text{sk}_0, \text{pk}_0) \leftarrow \text{KeyGen}(\text{par})$	
02 $(\text{sk}_1, \text{pk}_1) \leftarrow \text{KeyGen}(\text{par})$	
03 $b \xleftarrow{\$} \{0, 1\}$	
04 $(c^*, k^*) \leftarrow \text{Encap}(\text{pk}_b)$	$\parallel \mathbf{G}_0$
05 $k^* \xleftarrow{\$} \mathcal{K}$	$\parallel \mathbf{G}_1$
06 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\cdot, \cdot)}(\text{pk}_0, \text{pk}_1, (k^*, c^*))$	
07 return $\llbracket b = b' \rrbracket$	

Fig. 3. Games $\mathbf{G}_0$ - $\mathbf{G}_1$ for the proof of Lemma 1.

Game $\mathbf{G}_0$: This is the ANO-CCA security game for KEM.

$$\left| \Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \frac{1}{2} \right| = \text{Adv}_{\text{KEM}}^{\text{ANO-CCA}}(\mathcal{A}).$$

Game $\mathbf{G}_1$: In this game, instead of honestly computing k , we choose k uniformly at random. We can bound this change by reducing to the IND-CCA security of KEM. In the reduction, the simulator $\mathcal{A}_1^{\text{DECAPO}(\text{pk}, \cdot)}$ chooses the random bit b , let pk_b be the public key used in the IND-CCA security game, and generates

Reduction $\mathcal{A}_1^{\text{DECAPO}(\text{pk}, \cdot)}(\text{pk}, k^*, c^*)$	Oracle $\text{DECAPO}(\text{pk}_b, c)$
01 $b \xleftarrow{\$} \{0, 1\}$	06 if $c = c^*$
02 $\text{pk}_b := \text{pk}$	07 return $\perp$
03 $(\text{sk}_{1-b}, \text{pk}_{1-b}) \leftarrow \text{KeyGen}(\text{par})$	08 $k := \text{DECAPO}(\text{pk}, c)$
04 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\cdot, \cdot)}(\text{pk}_0, \text{pk}_1, (k^*, c^*))$	09 return k
05 return $\llbracket b = b' \rrbracket$	Oracle $\text{DECAPO}(\text{pk}_{1-b}, c)$
	10 if $c = c^*$
	11 return $\perp$
	12 $k := \text{Decap}(\text{sk}_{1-b}, c)$
	13 return k

Fig. 4. The adversary $\mathcal{A}_1^{\text{DECAPO}(\text{pk}, \cdot)}$ against the IND-CCA security of KEM for the proof of Lemma 1. In the reduction, $\mathcal{A}_1$ queries $\text{DECAPO}(\text{pk}, \cdot)$ at most q_D times.

another key pair $(\text{pk}_{1-b}, \text{sk}_{1-b})$, as described in Figure 4. When $\mathcal{A}$ queries the decapsulation oracle with (pk_b, c) , $\mathcal{A}_1^{\text{DECAPO}(\text{pk}, \cdot)}$ queries its own decapsulation oracle and returns the corresponding output. When $\mathcal{A}$ queries the decapsulation oracle with (pk_{1-b}, c) , the simulator uses sk_{1-b} to decapsulate.

$$\left| \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] \right| \leq \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{A}_1).$$

Finally, as shown in Figure 5, by choosing k uniformly random, we can reduce the success probability of $\mathcal{A}$ in $\mathbf{G}_1$ to the wANO-CCA security game for KEM, therefore

$$\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_{\text{KEM}}^{\text{wANO-CCA}}(\mathcal{A}_2) + \frac{1}{2}.$$

Reduction $\mathcal{A}_2^{\text{DECAPO}(\cdot, \cdot)}(\text{pk}_0, \text{pk}_1, c^*)$	Oracle $\text{DECAPO}(\text{pk}_0, c)$
01 $k^* \xleftarrow{\$} \mathcal{K}$	04 if $c = c^*$
02 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\cdot, \cdot)}(\text{pk}_0, \text{pk}_1, (k^*, c^*))$	05 return $\perp$
03 return b'	06 $k := \text{DECAPO}(\text{pk}_0, c)$
	07 return k
	Oracle $\text{DECAPO}(\text{pk}_1, c)$
	08 if $c = c^*$
	09 return $\perp$
	10 $k := \text{DECAPO}(\text{pk}_1, c)$
	11 return k

Fig. 5. The adversary $\mathcal{A}_2^{\text{DECAPO}(\cdot, \cdot)}$ against the wANO-CCA security of KEM for the proof of Lemma 1. In the reduction, $\mathcal{A}_2$ queries DECAPO^* at most q_D times.

Combining all the bounds above, we have Equation (5). $\square$

4 X-Wing and its Anonymity

The hybrid KEM X-Wing is built by combining the (post-quantum) ML-KEM and the ElGamal KEM. For readability, we generically view the ML-KEM as KEM_1 and the ElGamal KEM as KEM_2 , and we show the anonymity for this generic construction in Theorem 1. The concrete X-Wing is presented in Supp. Mat. A.3, and ML-KEM and the ElGamal KEM are in Supp. Mat. A.1 and A.2.

4.1 Generic Construction behind X-Wing

Essentially, X-Wing [5] follows the parallel KEM combiner proposed in [22]. As a hybrid KEM, it uses two KEMs $\text{KEM}_1 = (\text{KeyGen}_1, \text{Encap}_1, \text{Decap}_1)$ and $\text{KEM}_2 = (\text{KeyGen}_2, \text{Encap}_2, \text{Decap}_2)$ as black boxes, and combines them. We denote by HKEM the generic construction underlying X-Wing, and the description of HKEM is shown in Figure 6. Let $\mathcal{K}_1$ and $\mathcal{K}_2$ denote the key space of KEM_1 and KEM_2 respectively, and let $\mathcal{C}_2$ and $\mathcal{PK}_2$ denote the ciphertext space and the public key space of KEM_2 respectively. Let $\mathcal{L}$ be the label space for label and $\mathcal{K}$ be the key space for HKEM.

Algorithm KeyGen(par)	Algorithm Encap(pk)	Algorithm Decap(sk, c)
01 $(\text{sk}_1, \text{pk}_1) \leftarrow \text{KeyGen}_1(\text{par}_1)$	06 parse pk as $(\text{pk}_1, \text{pk}_2)$	12 parse sk as $(\text{sk}_1, \text{sk}_2, \text{pk}_2)$
02 $(\text{sk}_2, \text{pk}_2) \leftarrow \text{KeyGen}_2(\text{par}_2)$	07 $(k_1, c_1) \leftarrow \text{Encap}_1(\text{pk}_1)$	13 parse c as (c_1, c_2)
03 $\text{pk} := (\text{pk}_1, \text{pk}_2)$	08 $(k_2, c_2) \leftarrow \text{Encap}_2(\text{pk}_2)$	14 $k_1 := \text{Decap}_1(\text{sk}_1, c_1)$
04 $\text{sk} := (\text{sk}_1, \text{sk}_2, \text{pk}_2)$	09 $k := H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$	15 $k_2 := \text{Decap}_2(\text{sk}_2, c_2)$
05 return (pk, sk)	10 $c := (c_1, c_2)$	16 $k := H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$
	11 return (k, c)	17 return k

Fig. 6. Description of HKEM, as the generic construction behind X-Wing. According to the definition in [5], the label is encoded as 6-byte ASCII string ““\./^\””.

Remark 1 (Limitations of X-Wing and our variant in Section 5). As it was pointed out by Günther et al.[24], the anonymity of X-Wing requires both underlying KEMs to be anonymous simultaneously. This is because X-Wing runs both KEMs in parallel (cf. Line 09 in Figure 6) and concatenates their ciphertexts. Hence, as long as one ciphertext leaks which public key it belongs to, the anonymity of X-Wing is broken. Although we relax the requirement of the underlying KEMs to weak anonymity, the problem remains. The same holds true for our variant in Section 5 as well.

Unlike X-Wing and our variant, the construction in [24] is “sequential”, and it does not require both KEMs to be anonymous. More precisely, their construction requires the outer KEM to have statistically uniform ciphertexts (cf. Figures 1

and 4 in [24]). This property can be achieved by the ElGamal KEM⁴, and we use such a property as well (cf. Lemma 4). Hence, they claimed that their combiner is not fully generic. Their design uses several additional cryptographic primitives, including a pseudorandom generator (PRG) and a length-preserving symmetric encryption scheme (SE), and a split-key PRF. Besides the anonymity of one KEM and the PRF security, the anonymity of their hybrid KEM requires either the security of the PRG together with the one-time pseudorandomness of the SE ciphertext, or the strong uniformity (where the adversary has the secret key) of the ciphertext of the other KEM.

4.2 Anonymity of HKEM

The anonymity of HKEM requires KEM_2 to be wANO-PCA secure and in the case of X-Wing, KEM_2 is the ElGamal KEM that is unconditionally wANO-CCA* secure (cf. Lemma 4), which is stronger than wANO-PCA. Hence, it trivially satisfies this requirement.

After that, we distinguish if KEM_1 is further IND-CCA. If it is the case, the anonymity of HKEM requires wANO-CCA and WCPR-CCA of KEM_1 (cf. Theorem 1). Our security proof for this is memory-tight [4], and we adapt some techniques from [13, 21]. If KEM_1 is not IND-CCA, the anonymity of HKEM requires KEM_1 to be wANO-PCA (cf. Lemma 2). Although Lemma 2 only proves weak anonymity of HKEM, by Lemma 1 we can show HKEM is (standard) anonymous, if HKEM is IND-CCA secure, which is concluded in Lemma 3. If HKEM is X-Wing, then it has been proven to be IND-CCA secure in [5]. However, the proof in [5, Theorem 1] is not memory-tight. We use some techniques in Theorem 1 to avoid the memory loss in Theorem 7.

CASE I: KEM_1 IS IND-CCA SECURE. The following Theorem 1 proves the ANO-CCA of HKEM under the case when KEM_1 is IND-CCA. In this case, the only requirement for KEM_2 is wANO-PCA, and all the reductions are tight in both time and memory.

Theorem 1 (Anonymity of HKEM (Case I)). *Let $\text{KEM}_1 = (\text{KeyGen}_1, \text{Encap}_1, \text{Decap}_1)$ be a δ -correct key encapsulation mechanism. Let $\text{KEM}_2 = (\text{KeyGen}_2, \text{Encap}_2, \text{Decap}_2)$ be a wANO-PCA secure key encapsulation mechanism. Let $H : \mathcal{L} \times \mathcal{K}_1 \times \mathcal{K}_2 \times \mathcal{C}_2 \times \mathcal{PK}_2 \rightarrow \mathcal{K}$ be a random oracle and let $\mathcal{A}$ be an adversary against the ANO-CCA security of the hybrid KEM HKEM, making at most q_H queries to the random oracle and q_D queries to the decapsulation oracles. Then there exists an IND-CCA adversary $\mathcal{B}_1$, a WCPR-CCA adversary $\mathcal{B}_2$, and a wANO-CCA adversary $\mathcal{B}_3$ against KEM_1 , and a wANO-PCA adversary $\mathcal{C}$ against KEM_2 such that*

$$\begin{aligned} \text{Adv}_{\text{HKEM}}^{\text{ANO-CCA}}(\mathcal{A}) \leq & \text{Adv}_{\text{KEM}_1}^{\text{wANO-CCA}}(\mathcal{B}_3) + \text{Adv}_{\text{KEM}_1}^{\text{IND-CCA}}(\mathcal{B}_1) + \text{Adv}_{\text{KEM}_1}^{\text{WCPR-CCA}}(\mathcal{B}_2) \\ & + \text{Adv}_{\text{KEM}_2}^{\text{wANO-PCA}}(\mathcal{C}) + \delta + \frac{1 + 2q_H + 2q_D}{|\mathcal{K}_1|}, \end{aligned} \quad (6)$$

⁴ It is named as DHKEM in [24].

and

$$\begin{aligned}
\text{Time}(\mathcal{B}_1) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{B}_1) &\approx \text{Mem}(\mathcal{A}), \\
\text{Time}(\mathcal{B}_2) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{B}_2) &\approx \text{Mem}(\mathcal{A}), \\
\text{Time}(\mathcal{B}_3) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{B}_3) &\approx \text{Mem}(\mathcal{A}), \\
\text{Time}(\mathcal{C}) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{C}) &\approx \text{Mem}(\mathcal{A}).
\end{aligned}$$

PROOF. Let $\mathcal{A}$ be an adversary against the ANO-CCA security of HKEM. Consider the sequence of games $\mathbf{G}_0$ - $\mathbf{G}_7$ shown in Figure 7.

Game $\mathbf{G}_0$ - $\mathbf{G}_7$		Oracle $\text{DECAPO}(\text{pk}_b, c)$	
01 $(\text{sk}_{1,0}, \text{pk}_{1,0}) \leftarrow \text{KeyGen}_1(\text{par}_1)$		31 if $c = c^*$	
02 $(\text{sk}_{1,1}, \text{pk}_{1,1}) \leftarrow \text{KeyGen}_1(\text{par}_1)$		32 return $\perp$	
03 $(\text{sk}_{2,0}, \text{pk}_{2,0}) \leftarrow \text{KeyGen}_2(\text{par}_2)$		33 parse sk_b as $(\text{sk}_{1,b}, \text{sk}_{2,b})$	
04 $(\text{sk}_{2,1}, \text{pk}_{2,1}) \leftarrow \text{KeyGen}_2(\text{par}_2)$		34 parse c as (c_1, c_2)	
05 $\text{pk}_0 := (\text{pk}_{1,0}, \text{pk}_{2,0})$		35 if $c_1 = c_1^*$	$\parallel \mathbf{G}_1$ - $\mathbf{G}_7$
06 $\text{pk}_1 := (\text{pk}_{1,1}, \text{pk}_{2,1})$		36 $k_2 := \text{Decap}_2(\text{sk}_{2,b}, c_2)$	$\parallel \mathbf{G}_1$ - $\mathbf{G}_4$
07 $\text{sk}_0 := (\text{sk}_{1,0}, \text{sk}_{2,0})$		37 return $H(\text{label} \parallel k_1^* \parallel k_2 \parallel c_2 \parallel \text{pk}_{2,b})$	$\parallel \mathbf{G}_1$ - $\mathbf{G}_4$
08 $\text{sk}_1 := (\text{sk}_{1,1}, \text{sk}_{2,1})$		38 return $H_b(\text{label} \parallel c_1^* \parallel c_2 \parallel \text{pk}_{2,b})$	$\parallel \mathbf{G}_5$ - $\mathbf{G}_7$
09 $H_b, H_{1-b} \xleftarrow{\$} \Omega_H$	$\parallel \mathbf{G}_5$ - $\mathbf{G}_7$	39 $k_1 := \text{Decap}_1(\text{sk}_{1,b}, c_1)$	
10 $H' \xleftarrow{\$} \Omega_H$	$\parallel \mathbf{G}_6$ - $\mathbf{G}_7$	40 if $k_1 = k_1^* \vee k_1 = k_1'$	$\parallel \mathbf{G}_4$ - $\mathbf{G}_7$
11 $b \xleftarrow{\$} \{0, 1\}$		41 BAD $_1 := \text{true}; \text{abort}$	$\parallel \mathbf{G}_4$ - $\mathbf{G}_7$
12 $(k_1^*, c_1^*) \leftarrow \text{Encap}_1(\text{pk}_{1,b})$		42 $k_2 := \text{Decap}_2(\text{sk}_{2,b}, c_2)$	$\parallel \mathbf{G}_0$ - $\mathbf{G}_5$
13 $k_1^* \xleftarrow{\$} \mathcal{K}_1$	$\parallel \mathbf{G}_2$ - $\mathbf{G}_7$	43 $k = H(\text{label} \parallel k_1 \parallel k_2 \parallel c_2 \parallel \text{pk}_{2,b})$	$\parallel \mathbf{G}_0$ - $\mathbf{G}_5$
14 $k_1' \xleftarrow{\$} \mathcal{K}_1$	$\parallel \mathbf{G}_3$ - $\mathbf{G}_7$	44 $k = H'(\text{label} \parallel k_1 \parallel * \parallel c_2 \parallel \text{pk}_{2,b})$	$\parallel \mathbf{G}_6$ - $\mathbf{G}_7$
15 if $k_1^* = k_1'$	$\parallel \mathbf{G}_5$ - $\mathbf{G}_7$	45 return k	
16 BAD $_2 := \text{true}; \text{abort}$	$\parallel \mathbf{G}_5$ - $\mathbf{G}_7$	Oracle $\text{DECAPO}(\text{pk}_{1-b}, c)$	
17 $(k_2^*, c_2^*) \leftarrow \text{Encap}_2(\text{pk}_{2,b})$	$\parallel \mathbf{G}_0$ - $\mathbf{G}_6$	46 if $c = c^*$	
18 $(k_2^*, c_2^*) \leftarrow \text{Encap}_2(\text{pk}_{2,0})$	$\parallel \mathbf{G}_7$	47 return $\perp$	
19 $k^* := H(\text{label} \parallel k_1^* \parallel k_2^* \parallel c_2^* \parallel \text{pk}_{2,b})$	$\parallel \mathbf{G}_0$ - $\mathbf{G}_4$	48 parse sk_{1-b} as $(\text{sk}_{1,1-b}, \text{sk}_{2,1-b})$	
20 $k^* \xleftarrow{\$} \mathcal{K}$	$\parallel \mathbf{G}_5$ - $\mathbf{G}_7$	49 parse c as (c_1, c_2)	
21 $c^* := (c_1^*, c_2^*)$		50 if $c_1 = c_1^*$	$\parallel \mathbf{G}_3$ - $\mathbf{G}_7$
22 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\cdot, \cdot), H(\cdot)}(\text{pk}_0, \text{pk}_1, (c^*, k^*))$		51 $k_2 := \text{Decap}_2(\text{sk}_{2,1-b}, c_2)$	$\parallel \mathbf{G}_3$ - $\mathbf{G}_4$
23 return $[b = b']$		52 return $H(\text{label} \parallel k_1' \parallel k_2 \parallel c_2 \parallel \text{pk}_{2,1-b})$	$\parallel \mathbf{G}_3$ - $\mathbf{G}_4$
Oracle $H(\text{label} \parallel k_1 \parallel k_2 \parallel c_2 \parallel \text{pk}_2)$		53 return $H_{1-b}(\text{label} \parallel c_1^* \parallel c_2 \parallel \text{pk}_{2,1-b})$	$\parallel \mathbf{G}_5$ - $\mathbf{G}_7$
24 if $k_1 = k_1^* \vee k_1 = k_1'$	$\parallel \mathbf{G}_5$ - $\mathbf{G}_7$	54 $k_1 := \text{Decap}_1(\text{sk}_{1,1-b}, c_1)$	
25 BAD $_3 := \text{true}; \text{abort}$	$\parallel \mathbf{G}_5$ - $\mathbf{G}_7$	55 if $k_1 = k_1^* \vee k_1 = k_1'$	$\parallel \mathbf{G}_4$ - $\mathbf{G}_7$
26 if $\text{pk}_2 = \text{pk}_{2,0} \wedge \text{Decap}(\text{sk}_{2,0}, c_2) = k_2$	$\parallel \mathbf{G}_6$ - $\mathbf{G}_7$	56 BAD $_1 := \text{true}; \text{abort}$	$\parallel \mathbf{G}_4$ - $\mathbf{G}_7$
27 return $H'(\text{label} \parallel k_1 \parallel * \parallel c_2 \parallel \text{pk}_2)$	$\parallel \mathbf{G}_6$ - $\mathbf{G}_7$	57 $k_2 := \text{Decap}_2(\text{sk}_{2,1-b}, c_2)$	$\parallel \mathbf{G}_0$ - $\mathbf{G}_5$
28 if $\text{pk}_2 = \text{pk}_{2,1} \wedge \text{Decap}(\text{sk}_{2,1}, c_2) = k_2$	$\parallel \mathbf{G}_6$ - $\mathbf{G}_7$	58 $k = H(\text{label} \parallel k_1 \parallel k_2 \parallel c_2 \parallel \text{pk}_{2,1-b})$	$\parallel \mathbf{G}_0$ - $\mathbf{G}_5$
29 return $H'(\text{label} \parallel k_1 \parallel * \parallel c_2 \parallel \text{pk}_2)$	$\parallel \mathbf{G}_6$ - $\mathbf{G}_7$	59 $k = H'(\text{label} \parallel k_1 \parallel * \parallel c_2 \parallel \text{pk}_{2,1-b})$	$\parallel \mathbf{G}_6$ - $\mathbf{G}_7$
30 return $H(\text{label} \parallel k_1 \parallel k_2 \parallel c_2 \parallel \text{pk}_2)$		60 return k	

Fig. 7. Games $\mathbf{G}_0$ - $\mathbf{G}_7$ for the proof of Theorem 1.

Game $\mathbf{G}_0$: This is the ANO-CCA security game for HKEM.

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] = \text{Adv}_{\text{HKEM}}^{\text{ANO-CCA}}(\mathcal{A}) + \frac{1}{2}.$$

Game $\mathbf{G}_1$: In the decapsulation oracle $\text{DECAPO}(\text{pk}_b, \cdot)$ of this game, if $c = (c_1^*, c_2)$, then we use k_1^* instead of $\text{Decap}(\text{sk}_{1,b}, c_1^*)$ to decapsulate the ciphertext, where $(k_1^*, c_1^*) \leftarrow \text{Encap}_1(\text{pk}_b)$.

Reduction $\mathcal{B}_1^{\text{DECAPO}(\text{pk}, \cdot)}(\text{pk}, k_1^*, c_1^*)$	Oracle $\text{DECAPO}(\text{pk}_b, c)$
01 $b \xleftarrow{\$} \{0, 1\}$	14 if $c = c^*$
02 $\text{pk}_{1,b} := \text{pk}$	15 return $\perp$
03 $(\text{sk}_{1,1-b}, \text{pk}_{1,1-b}) \leftarrow \text{KeyGen}_1(\text{par}_1)$	16 parse c as (c_1, c_2)
04 $(\text{sk}_{1,0}, \text{pk}_{1,0}) \leftarrow \text{KeyGen}_2(\text{par}_2)$	17 if $c_1 = c_1^*$
05 $(\text{sk}_{2,1}, \text{pk}_{2,1}) \leftarrow \text{KeyGen}_2(\text{par}_2)$	18 $k_2 := \text{Decap}_2(\text{sk}_{2,b}, c_2)$
06 $\text{pk}_0 := (\text{pk}_{1,0}, \text{pk}_{2,0})$	19 return $H(\text{label} \ k_1^* \ k_2 \ c_2 \ \text{pk}_{2,b})$
07 $\text{pk}_1 := (\text{pk}_{1,1}, \text{pk}_{2,1})$	20 $k_1 := \text{DECAPO}(\text{pk}, c_1)$
08 $(k_2^*, c_2^*) \leftarrow \text{Encap}_2(\text{pk}_{2,b})$	21 $k_2 := \text{Decap}_2(\text{sk}_{2,b}, c_2)$
09 $k^* := H(\text{label} \ k_1^* \ k_2^* \ c_2^* \ \text{pk}_{2,b})$	22 $k = H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_{2,b})$
10 $c^* := (c_1^*, c_2^*)$	23 return k
11 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\cdot, \cdot), H(\cdot)}(\text{pk}_0, \text{pk}_1, (c^*, k^*))$	Oracle $\text{DECAPO}(\text{pk}_{1-b}, c)$
12 return $\llbracket b = b' \rrbracket$	24 if $c = c^*$
Oracle $H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$	25 return $\perp$
13 return $H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$	26 parse c as (c_1, c_2)
	27 $k_1 := \text{Decap}_1(\text{sk}_{1,1-b}, c_1)$
	28 $k_2 := \text{Decap}_2(\text{sk}_{2,1-b}, c_2)$
	29 $k = H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_{2,1-b})$
	30 return k

Fig. 8. The adversary $\mathcal{B}_1^{\text{DECAPO}(\text{pk}, \cdot)}$ against the IND-CCA security of KEM_1 for the proof of Theorem 1. In the reduction, $\mathcal{B}_1$ queries DECAPO at most q_D times.

Since KEM_1 is δ -correct, we can bound the difference of $\mathbf{G}_1$ and $\mathbf{G}_0$ by

$$\left| \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] \right| \leq \delta.$$

Game $\mathbf{G}_2$: In this game, instead of getting k_1^* by computing $(k_1^*, c_1^*) \leftarrow \text{Encap}_1(\text{pk}_b)$, we choose a uniformly random $k_1^* \xleftarrow{\$} \mathcal{K}_1$. As shown in Figure 8, we can bound this change by the IND-CCA advantage of KEM_1 .

After receiving the challenge $(\text{pk}, k_1^*, c_1^*)$, the simulator $\mathcal{B}_1$ chooses a random bit b and lets pk be the challenge public key of KEM_1 (i.e. $\text{pk}_{1,b}$). Then $\mathcal{B}_1$ generates another key pair $(\text{pk}_{1,1-b}, \text{sk}_{1,1-b})$ for KEM_1 and two different key pairs for KEM_2 . Therefore, $\mathcal{B}_1$ has all the secret keys except $\text{sk}_{1,b}$ for KEM_1 . Thus in the decapsulation oracles, $\mathcal{B}_1$ can use these secret keys and its oracle DECAPO to answer the queries from $\mathcal{A}$, except for the case when $\mathcal{A}$ issues a query (c_1^*, c_2) to $\text{DECAPO}(\text{pk}_b, \cdot)$. In the latter case, $\mathcal{B}_1$ first uses $\text{sk}_{2,b}$ to get $k_2 := \text{Decap}(\text{sk}_{2,b}, c_2)$, and then outputs $H(\text{label} \| k_1^* \| k_2 \| c_2 \| \text{pk}_{2,b})$ as the corresponding session key. Therefore,

$$\left| \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \right| \leq 2 \cdot \text{Adv}_{\text{KEM}_1}^{\text{IND-CCA}}(\mathcal{B}_1).$$

Game $\mathbf{G}_3$: In this game, if the adversary $\mathcal{A}$ queries the decapsulation oracle $\text{DECAPO}(\text{pk}_{1-b}, \cdot)$ with $c = (c_1^*, c_2)$, where c_1^* is the encapsulation cipher-

Reduction $\mathcal{B}_2^{\text{DECAPO}(\cdot, \cdot)}(\text{pk}_{1,0}, \text{pk}_{1,1}, b, c_1^*, k_1')$	Oracle $\text{DECAPO}(\text{pk}_b, c)$
01 $(\text{sk}_{2,0}, \text{pk}_{2,0}) \leftarrow \text{KeyGen}_2(\text{par}_2)$	12 if $c = c^*$
02 $(\text{sk}_{2,1}, \text{pk}_{2,1}) \leftarrow \text{KeyGen}_2(\text{par}_2)$	13 return $\perp$
03 $\text{pk}_0 := (\text{pk}_{1,0}, \text{pk}_{2,0})$	14 parse c as (c_1, c_2)
04 $\text{pk}_1 := (\text{pk}_{1,1}, \text{pk}_{2,1})$	15 if $c_1 = c_1^*$
05 $(k_2^*, c_2^*) \leftarrow \text{Encap}_2(\text{pk}_{2,b})$	16 $k_2 := \text{Decap}_2(\text{sk}_{2,b}, c_2)$
06 $k_1^* \xleftarrow{\$} \mathcal{K}_1$	17 return $H(\text{label} \ k_1^* \ k_2 \ c_2 \ \text{pk}_{2,b})$
07 $k^* := H(\text{label} \ k_1^* \ k_2^* \ c_2^* \ \text{pk}_{2,b})$	18 $k_1 := \text{DECAPO}(\text{pk}_{1,b}, c_1)$
08 $c^* := (c_1^*, c_2^*)$	19 $k_2 := \text{Decap}_2(\text{sk}_{2,b}, c_2)$
09 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\cdot, \cdot), H(\cdot)}(\text{pk}_0, \text{pk}_1, (c^*, k^*))$	20 $k = H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_{2,b})$
10 return $[b = b']$	21 return k
Oracle $H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$	Oracle $\text{DECAPO}(\text{pk}_{1-b}, c)$
11 return $H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$	22 if $c = c^*$
	23 return $\perp$
	24 parse c as (c_1, c_2)
	25 if $c_1 = c_1^*$
	26 $k_2 := \text{Decap}_2(\text{sk}_{2,b}, c_2)$
	27 return $H(\text{label} \ k_1' \ k_2 \ c_2 \ \text{pk}_{2,1-b})$
	28 $k_1 := \text{DECAPO}(\text{pk}_{1,1-b}, c_1)$
	29 $k_2 := \text{Decap}_2(\text{sk}_{2,b}, c_2)$
	30 $k = H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_{2,1-b})$
	31 return k

Fig. 9. The adversary $\mathcal{B}_2^{\text{DECAPO}(\cdot, \cdot)}$ against the WCPR-CCA security of KEM_1 for the proof of Theorem 1. In the reduction, $\mathcal{B}_2$ queries DECAPO at most q_D times.

text of $\text{Encap}_1(\text{pk}_{1,b})$, then we use a uniformly random $k_1' \xleftarrow{\$} \mathcal{K}_1$ instead of $\text{Decap}(\text{sk}_{1,1-b}, c_1^*)$ to decapsulate the ciphertext.

If $\text{Decap}_1(\text{sk}_{1,1-b}, c_1^*)$ is indistinguishable from a random session key k_1' , then $\mathcal{A}$'s view in $\mathbf{G}_2$ and $\mathbf{G}_3$ are the same. Thus by using the reduction shown in Figure 9, we can bound the difference of $\mathbf{G}_2$ and $\mathbf{G}_3$ by the WCPR-CCA advantage of KEM_1 and obtain

$$\left| \Pr[\mathbf{G}_3^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] \right| \leq \text{Adv}_{\text{KEM}_1}^{\text{WCPR-CCA}}(\mathcal{B}_2).$$

Game $\mathbf{G}_4$: In this game, if $\mathcal{A}$ queries the decapsulation oracles with $(c_1 \neq c_1^*, c_2)$ such that $\text{Decap}_1(\cdot, c_1)$ equals to k_1^* or k_1' , then we abort the game. We define this event as BAD_1 , and when BAD_1 never happens, the distribution remains the same. Since both k_1^* or k_1' are chosen uniformly random and hidden from the adversary $\mathcal{A}$, we have

$$\left| \Pr[\mathbf{G}_4^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_3^{\mathcal{A}} \Rightarrow 1] \right| \leq \Pr[\text{BAD}_1] \leq 2 \cdot \frac{q_D}{|\mathcal{K}_1|}.$$

Game $\mathbf{G}_5$: In this game, we choose k^* uniformly random and modify the decapsulation value of $\text{DECAPO}(\cdot, (c_1^*, c_2))$ by reprogramming the random oracle.

Specifically speaking, in the decapsulation oracles $\text{DECAPO}(\cdot, \cdot)$, for the random session keys k_1^* and k'_1 , we replace the hash values $H(\text{label} \| k_1^* \| k_2 \| c_2 \| \text{pk}_{2,b})$ and $H(\text{label} \| k'_1 \| k_2 \| c_2 \| \text{pk}_{2,1-b})$ with $H_b(\text{label} \| c_1^* \| c_2 \| \text{pk}_{2,b})$ and $H_{1-b}(\text{label} \| c_1^* \| c_2 \| \text{pk}_{2,1-b})$ respectively, where H_b and H_{1-b} are random functions. If $k_1^* \neq k'_1$ and $\mathcal{A}$ never queries the random oracle H with an input containing k_1^* or k'_1 , then $\mathcal{A}$'s view in $\mathbf{G}_4$ and $\mathbf{G}_5$ are the same because $\mathcal{A}$ is not allowed to query (c_1^*, c_2^*) to the decapsulation oracles.

We define the event BAD_2 to denote $k_1^* = k'_1$, and define the event BAD_3 occurs when $\mathcal{A}$ queries H with $(\text{label} \| k_1 \| k_2 \| c_2 \| \text{pk}_2)$ where $k_1 = k_1^* \vee k'_1$. Since both k_1^* and k'_1 are chosen uniformly random and hidden from the adversary $\mathcal{A}$, we have

$$\left| \Pr[\mathbf{G}_5^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_4^{\mathcal{A}} \Rightarrow 1] \right| \leq \Pr[\text{BAD}_2] + \Pr[\text{BAD}_3] \leq \frac{1}{|\mathcal{K}_1|} + 2 \cdot \frac{q_H}{|\mathcal{K}_1|}.$$

Reduction $\mathcal{C}^{\text{PCO}_2(\cdot, \cdot, \cdot)}(\text{pk}_{2,0}, \text{pk}_{2,1}, c_2^*)$	Oracle $\text{DECAPO}(\text{pk}_0, c)$
01 $(\text{sk}_{1,0}, \text{pk}_{1,0}) \leftarrow \text{KeyGen}_1(\text{par}_1)$	23 if $c = c^*$
02 $(\text{sk}_{1,1}, \text{pk}_{1,1}) \leftarrow \text{KeyGen}_1(\text{par}_1)$	24 return $\perp$
03 $\text{pk}_0 := (\text{pk}_{1,0}, \text{pk}_{2,0})$	25 parse c as (c_1, c_2)
04 $\text{pk}_1 := (\text{pk}_{1,1}, \text{pk}_{2,1})$	26 if $c_1 = c_1^*$
05 $H_b, H_{1-b} \xleftarrow{\$} \Omega_H$	27 return $H_0(\text{label} \ c_1^* \ c_2 \ \text{pk}_{2,0})$
06 $H' \xleftarrow{\$} \Omega_H$	28 $k_1 := \text{Decap}_1(\text{sk}_{1,0}, c_1)$
07 $(k_1^*, c_1^*) \leftarrow \text{Encap}_1(\text{pk}_{1,1})$	29 if $k_1 = k_1^* \vee k_1 = k'_1$
08 $k'_1 \xleftarrow{\$} \mathcal{K}_1$	30 BAD ₁ := true ; abort
09 $k_1^* \xleftarrow{\$} \mathcal{K}_1$	31 $k = H'(\text{label} \ k_1 \ \star \ c_2 \ \text{pk}_{2,0})$
10 if $k_1^* = k'_1$	32 return k
11 BAD ₂ := true ; abort	Oracle $\text{DECAPO}(\text{pk}_1, c)$
12 $k^* \xleftarrow{\$} \mathcal{K}$	33 if $c = c^*$
13 $c^* := (c_1^*, c_2^*)$	34 return $\perp$
14 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\cdot, \cdot, H(\cdot))}(\text{pk}_0, \text{pk}_1, (c^*, k^*))$	35 parse c as (c_1, c_2)
15 return b'	36 if $c_1 = c_1^*$
Oracle $H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$	37 return $H_1(\text{label} \ c_1^* \ c_2 \ \text{pk}_{2,1})$
16 if $k_1 = k_1^* \vee k_1 = k'_1$	38 $k_1 := \text{Decap}_1(\text{sk}_{1,1}, c_1)$
17 BAD ₃ := true ; abort	39 if $k_1 = k_1^* \vee k_1 = k'_1$
18 if $\text{pk}_2 = \text{pk}_{2,0} \wedge \text{PCA}_2(\text{pk}_{2,0}, k_2, c_2) = 1$	40 BAD ₁ := true ; abort
19 return $H'(\text{label} \ k_1 \ \star \ c_2 \ \text{pk}_2)$	41 $k = H'(\text{label} \ k_1 \ \star \ c_2 \ \text{pk}_{2,1})$
20 if $\text{pk}_2 = \text{pk}_{2,1} \wedge \text{PCA}_2(\text{pk}_{2,1}, k_2, c_2) = 1$	42 return k
21 return $H'(\text{label} \ k_1 \ \star \ c_2 \ \text{pk}_2)$	
22 return $H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$	

Fig. 10. The adversary $\mathcal{C}^{\text{PCO}_2(\cdot, \cdot, \cdot)}(\text{pk}_{2,0}, \text{pk}_{2,1}, c_2^*)$ against the wANO-PCA security of KEM_2 for the proof of Theorem 1. In the reduction, $\mathcal{C}$ queries PCO_2 at most $2q_H$ times.

Game $\mathbf{G}_6$: In this game, instead of using the secret keys of KEM_2 to recover k_2 from c_2 in the decapsulation oracles $\text{DECAPO}(\cdot, \cdot)$, we simply return $H'(\text{label} \| k_1 \| \star$

$\|c_2\|\text{pk}_2$) as the decapsulation value of (c_1, c_2) , where H' is a random function. By doing this, we no longer need the secret key of KEM_2 in the decapsulation oracles.

In order to keep the consistency, for each query $(\text{label}\|k_1\|k_2\|c_2\|\text{pk}_2)$ to the random oracle H , if (k_2, c_2, pk_2) is a valid pair (i.e. there exists a bit $\hat{b}$ such that $\text{Decap}_2(\text{sk}_{2,\hat{b}}, c_2) = k_2$ and $\text{pk}_{2,\hat{b}} = \text{pk}_2$), then we reprogram the corresponding hash value as $H'(\text{label}\|k_1\| \star \|c_2\|\text{pk}_2)$. Since there is a bijection between (c_2, pk_2) and (k_2, c_2, pk_2) , this modification does not change the distribution, thus we have

$$\Pr[\mathbf{G}_6^A \Rightarrow 1] = \Pr[\mathbf{G}_5^A \Rightarrow 1].$$

Reduction $\mathcal{B}_3^{\text{DECAPO}(\cdot, \cdot)}(\text{pk}_{1,0}, \text{pk}_{1,1}, c_1^*)$	Oracle $\text{DECAPO}(\text{pk}_0, c)$
01 $(\text{sk}_{2,0}, \text{pk}_{2,0}) \leftarrow \text{KeyGen}_2(\text{par}_2)$	23 if $c = c^*$
02 $(\text{sk}_{2,1}, \text{pk}_{2,1}) \leftarrow \text{KeyGen}_2(\text{par}_2)$	24 return $\perp$
03 $\text{pk}_0 := (\text{pk}_{1,0}, \text{pk}_{2,0})$	25 parse c as (c_1, c_2)
04 $\text{pk}_1 := (\text{pk}_{1,1}, \text{pk}_{2,1})$	26 if $c_1 = c_1^*$
05 $H_b, H_{1-b} \xleftarrow{\$} \Omega_H$	27 return $H_0(\text{label}\ c_1^*\ c_2\ \text{pk}_{2,0})$
06 $H' \xleftarrow{\$} \Omega_H$	28 $k_1 := \text{DECAPO}(\text{pk}_{1,0}, c_1)$
07 $k'_1 \xleftarrow{\$} \mathcal{K}_1$	29 if $k_1 = k_1^* \vee k_1 = k'_1$
08 $k_1^* \xleftarrow{\$} \mathcal{K}_1$	30 $\text{BAD}_1 := \text{true}; \text{abort}$
09 if $k_1^* = k'_1$	31 $k = H'(\text{label}\ k_1\ \star \ c_2\ \text{pk}_{2,0})$
10 $\text{BAD}_2 := \text{true}; \text{abort}$	32 return k
11 $(k_2^*, c_2^*) \leftarrow \text{Encap}_2(\text{pk}_{2,0})$	Oracle $\text{DECAPO}(\text{pk}_1, c)$
12 $k^* \xleftarrow{\$} \mathcal{K}$	33 if $c = c^*$
13 $c^* := (c_1^*, c_2^*)$	34 return $\perp$
14 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\cdot, \cdot), H(\cdot)}(\text{pk}_0, \text{pk}_1, (c^*, k^*))$	35 parse c as (c_1, c_2)
15 return b'	36 if $c_1 = c_1^*$
Oracle $H(\text{label}\ k_1\ k_2\ c_2\ \text{pk}_2)$	37 return $H_1(\text{label}\ c_1^*\ c_2\ \text{pk}_{2,1})$
16 if $k_1 = k_1^* \vee k_1 = k'_1$	38 $k_1 := \text{DECAPO}(\text{pk}_{1,1}, c_1)$
17 $\text{BAD}_3 := \text{true}; \text{abort}$	39 if $k_1 = k_1^* \vee k_1 = k'_1$
18 if $\text{pk}_2 = \text{pk}_{2,0} \wedge \text{Decap}(\text{sk}_{2,0}, c_2) = k_2$	40 $\text{BAD}_1 := \text{true}; \text{abort}$
19 return $H'(\text{label}\ k_1\ \star \ c_2\ \text{pk}_2)$	41 $k = H'(\text{label}\ k_1\ \star \ c_2\ \text{pk}_{2,1})$
20 if $\text{pk}_2 = \text{pk}_{2,1} \wedge \text{Decap}(\text{sk}_{2,1}, c_2) = k_2$	42 return k
21 return $H'(\text{label}\ k_1\ \star \ c_2\ \text{pk}_2)$	
22 return $H(\text{label}\ k_1\ k_2\ c_2\ \text{pk}_2)$	

Fig. 11. The adversary $\mathcal{B}_3^{\text{DECAPO}(\cdot, \cdot)}$ against the wANO-CCA security of KEM_1 for the proof of Theorem 1. In the reduction, $\mathcal{B}_3$ queries DECAPO at most q_D times.

Game $\mathbf{G}_7$: In this game, instead of using the random bit b to get c_2^* , we compute it by fixing the bit to be 0, that is $(k_2^*, c_2^*) \leftarrow \text{Encap}_2(\text{pk}_{2,0})$. Games $\mathbf{G}_7$ and $\mathbf{G}_6$ will only have a difference when $b \xleftarrow{\$} \{0, 1\}$ is chosen to be 1. By using the

reduction shown in Figure 10, we can bound this difference by the wANO-PCA advantage of KEM_2 :

$$\begin{aligned} \left| \Pr[\mathbf{G}_7^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_6^{\mathcal{A}} \Rightarrow 1] \right| &= \frac{1}{2} \cdot \left| \Pr[\mathbf{G}_7^{\mathcal{A}} \Rightarrow 1 | b = 1] - \Pr[\mathbf{G}_6^{\mathcal{A}} \Rightarrow 1 | b = 1] \right| \\ &\leq \text{Adv}_{\text{KEM}_2}^{\text{wANO-PCA}}(\mathcal{C}). \end{aligned}$$

Finally, as shown in Figure 11, we can bound the success probability of $\mathcal{A}$ in $\mathbf{G}_7$ by the wANO-CCA advantage of KEM_1 . In the reduction, after receiving the challenge $(\text{pk}_{1,0}, \text{pk}_{1,1}, c_1^*)$, $\mathcal{B}_3$ chooses a random $k^* \xleftarrow{\$} \mathcal{K}$ and generates two key pairs of KEM_2 . Then $\mathcal{B}_3$ computes c_2^* by $\text{Encap}_2(\text{pk}_{2,0})$, and sends $((\text{pk}_{1,0}, \text{pk}_{2,0}), (\text{pk}_{1,1}, \text{pk}_{2,1}), (c_1^*, c_2^*), k^*)$ to $\mathcal{A}$ as the challenge input. For any bit $\hat{b} \in \{0, 1\}$, if $\mathcal{A}$ queries the decapsulation oracle $\text{DECAPO}(\text{pk}_{\hat{b}}, \cdot)$ with $c = (c_1^*, c_2)$, then $\mathcal{B}_3$ returns $k = H_{\hat{b}}(\text{label} \| c_1^* \| c_2 \| \text{pk}_{2,\hat{b}})$ as the decapsulation value of c , where $H_{\hat{b}}$ are random functions chosen by $\mathcal{B}_3$. For any other $\mathcal{A}$'s queries to the decapsulation oracle $\text{DECAPO}(\text{pk}_{\hat{b}}, \cdot)$, $\mathcal{B}_3$ can use his own decapsulation oracle $\text{DECAPO}(\text{pk}_{2,\hat{b}}, \cdot)$ to recover k_1 , and then returns $k = H'(\text{label} \| k_1 \| \star \| c_2 \| \text{pk}_{2,\hat{b}})$. In the meantime, for each query $(\text{label} \| k_1 \| k_2 \| c_2 \| \text{pk}_2)$ to the random oracle H , $\mathcal{B}_3$ uses the corresponding secret keys of KEM_2 (which are generated by himself) to justify whether $k_2 = \text{Decap}_2(\text{sk}_{2,\hat{b}}, c_2)$ and to keep the consistency with the decapsulation oracles. This perfectly simulates $\mathcal{A}$'s view in game $\mathbf{G}_7$ and we have

$$\Pr[\mathbf{G}_7^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_{\text{KEM}_1}^{\text{wANO-CCA}}(\mathcal{B}_3) + \frac{1}{2}.$$

We have Equation (6) by combining all the bounds above. $\square$

CASE II: KEM_1 IS NOT IND-CCA SECURE. Now we consider the case that KEM_1 may not be IND-CCA secure. We first use the following Lemma 2 to show the weak anonymity of HKEM, which requires the wANO-PCA of both KEM_1 and KEM_2 . Since the ciphertext of HKEM simply concatenates the ciphertexts of KEM_1 and KEM_2 , it is impossible to achieve the weak anonymity of the hybrid KEM if any ciphertext of these two underlying KEMs reveals the challenge bit in the anonymity game, which has been observed in [24] as well. Therefore, requiring both KEMs to be wANO-PCA is inherent. However, we need some memory to record the decapsulation values and hash queries in order to keep the consistency, and thus the reductions are not memory-tight. We note that adding the ciphertext of KEM_1 into the hash can avoid this memory loss, and we give formal treatments of this in Section 5.

Lemma 2 (wANO-CCA* of HKEM). *Let $\text{KEM}_1 = (\text{KeyGen}_1, \text{Encap}_1, \text{Decap}_1)$ and $\text{KEM}_2 = (\text{KeyGen}_2, \text{Encap}_2, \text{Decap}_2)$ be two wANO-PCA secure key encapsulation mechanisms. Let $H : \mathcal{L} \times \mathcal{K}_1 \times \mathcal{K}_2 \times \mathcal{C}_2 \times \mathcal{PK}_2 \rightarrow \mathcal{K}$ be a random oracle and let $\mathcal{A}$ be an adversary against the wANO-CCA* security of HKEM, making at most q_H queries to the random oracle and q_D queries to the decapsulation oracles.*

Then there exists a wANO-PCA adversary $\mathcal{C}$ against KEM_1 and a wANO-PCA adversary $\mathcal{B}$ against KEM_2 such that

$$\text{Adv}_{\text{HKEM}}^{\text{wANO-CCA}^*}(\mathcal{A}) \leq \text{Adv}_{\text{KEM}_1}^{\text{wANO-PCA}}(\mathcal{C}) + \text{Adv}_{\text{KEM}_2}^{\text{wANO-PCA}}(\mathcal{B}), \quad (7)$$

and

$$\begin{aligned} \text{Time}(\mathcal{B}) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{B}) &= \text{Mem}(\mathcal{A}) + \log |\mathcal{L}_H|, \\ \text{Time}(\mathcal{C}) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{C}) &= \text{Mem}(\mathcal{A}) + \log |\mathcal{L}_H| + \log |D|, \end{aligned}$$

where $\mathcal{L}_H$ and D denote the lists used to simulate the random oracle and the decapsulation oracle respectively, and $|\mathcal{L}_H| = \mathcal{O}(q_H)$, $|D| = \mathcal{O}(q_D)$.

PROOF. Let $\mathcal{A}$ be an adversary against the wANO-CCA* security of HKEM.

Game $\mathbf{G_0 - G_3}$	Oracle $\text{DECAPO}^*(\text{pk}_b, c)$
01 $(\text{sk}_{1,0}, \text{pk}_{1,0}) \leftarrow \text{KeyGen}_1(\text{par}_1)$	33 parse c as (c_1, c_2)
02 $(\text{sk}_{1,1}, \text{pk}_{1,1}) \leftarrow \text{KeyGen}_1(\text{par}_1)$	34 $k_1 := \text{Decap}_1(\text{sk}_{1,b}, c_1)$ $\parallel \mathbf{G_0-G_2}$
03 $(\text{sk}_{2,0}, \text{pk}_{2,0}) \leftarrow \text{KeyGen}_2(\text{par}_2)$	35 $k_2 := \text{Decap}_2(\text{sk}_{2,b}, c_2)$ $\parallel \mathbf{G_0}$
04 $(\text{sk}_{2,1}, \text{pk}_{2,1}) \leftarrow \text{KeyGen}_2(\text{par}_2)$	36 $k := H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_{2,b})$ $\parallel \mathbf{G_0}$
05 $\text{pk}_0 := (\text{pk}_{1,0}, \text{pk}_{2,0})$	37 $k := H'(\text{label}, k_1, c_2, \text{pk}_{2,b})$ $\parallel \mathbf{G_1-G_2}$
06 $\text{pk}_1 := (\text{pk}_{1,1}, \text{pk}_{2,1})$	38 $k_2 := \text{Decap}_2(\text{sk}_{2,b}, c_2)$ $\parallel \mathbf{G_3}$
07 $\text{sk}_0 := (\text{sk}_{1,0}, \text{sk}_{2,0})$	39 if $\exists (\text{label}, k_1, k_2, c_2, \text{pk}_{2,b}) \in \mathcal{L}_H$ s.t.
08 $\text{sk}_1 := (\text{sk}_{1,1}, \text{sk}_{2,1})$	40 $\text{Decap}_1(\text{sk}_{1,b}, c_1) = k_1$ $\parallel \mathbf{G_3}$
09 $\mathcal{L}_H := \emptyset$	41 return $\mathcal{L}_H[\text{label}, k_1, k_2, c_2, \text{pk}_{2,b}]$ $\parallel \mathbf{G_3}$
10 $H' \xleftarrow{\$} \Omega_{H'}$ $\parallel \mathbf{G_1-G_2}$	42 else if $D[\text{pk}_{2,b}, c_1, c_2] = \perp$ $\parallel \mathbf{G_3}$
11 $D := \emptyset$ $\parallel \mathbf{G_3}$	43 $k \xleftarrow{\$} \mathcal{K}$ $\parallel \mathbf{G_3}$
12 $b \xleftarrow{\$} \{0, 1\}$	44 $D := D \cup \{(\text{pk}_{2,b}, c_1, c_2)\}$ $\parallel \mathbf{G_3}$
13 $(k_1^*, c_1^*) \leftarrow \text{Encap}_1(\text{pk}_{1,b})$	45 $D[\text{pk}_{2,b}, c_1, c_2] := k$ $\parallel \mathbf{G_3}$
14 $(k_2^*, c_2^*) \leftarrow \text{Encap}_2(\text{pk}_{2,b})$ $\parallel \mathbf{G_0-G_1}$	46 $k := D[\text{pk}_{2,b}, c_1, c_2]$ $\parallel \mathbf{G_3}$
15 $(k_2^*, c_2^*) \leftarrow \text{Encap}_2(\text{pk}_{2,0})$ $\parallel \mathbf{G_2-G_3}$	47 return k
16 $c^* := (c_1^*, c_2^*)$	
17 $b' \leftarrow \mathcal{A}^{\text{DECAPO}^*(\cdot, \cdot), H(\cdot)}(\text{pk}_0, \text{pk}_1, c^*)$	Oracle $\text{DECAPO}^*(\text{pk}_{1-b}, c)$
18 return $\llbracket b = b' \rrbracket$	48 parse c as (c_1, c_2)
Oracle $H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$	49 $k_1 := \text{Decap}_1(\text{sk}_{1,1-b}, c_1)$ $\parallel \mathbf{G_0-G_2}$
19 if $\exists (\text{pk}_{2,\hat{b}}, c_1, c_2) \in D$ s.t.	50 $k_2 := \text{Decap}_2(\text{sk}_{2,1-b}, c_2)$ $\parallel \mathbf{G_0}$
20 $\text{pk}_2 = \text{pk}_{2,\hat{b}}$ $\parallel \mathbf{G_3}$	51 $k := H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_{2,1-b})$ $\parallel \mathbf{G_0}$
21 if $\text{Decap}_1(\text{sk}_{1,\hat{b}}, c_1) = k_1 \wedge$	52 $k := H'(\text{label}, k_1, c_2, \text{pk}_{2,1-b})$ $\parallel \mathbf{G_1-G_2}$
22 $\text{Decap}_2(\text{sk}_{2,\hat{b}}, c_2) = k_2$ $\parallel \mathbf{G_3}$	53 $k_2 := \text{Decap}_2(\text{sk}_{2,1-b}, c_2)$ $\parallel \mathbf{G_3}$
23 return $D[\text{pk}_{2,\hat{b}}, c_1, c_2]$ $\parallel \mathbf{G_3}$	54 if $\exists (\text{label}, k_1, k_2, c_2, \text{pk}_{2,1-b}) \in \mathcal{L}_H$ s.t.
24 if $\exists \hat{b} \in \{0, 1\}$ s.t. $\text{pk}_2 = \text{pk}_{2,\hat{b}} \wedge$	55 $\text{Decap}_1(\text{pk}_{1,1-b}, c_1) = k_1$ $\parallel \mathbf{G_3}$
25 $\text{Decap}_2(\text{sk}_{2,\hat{b}}, c_2) = k_2$ $\parallel \mathbf{G_1-G_2}$	56 return $\mathcal{L}_H[\text{label}, k_1, k_2, c_2, \text{pk}_{2,1-b}]$ $\parallel \mathbf{G_3}$
26 $\mathcal{L}_H[\text{label}, k_1, k_2, c_2, \text{pk}_2] :=$	57 else if $D[\text{pk}_{2,1-b}, c_1, c_2] = \perp$ $\parallel \mathbf{G_3}$
27 $H'(\text{label}, k_1, k_2, c_2, \text{pk}_2)$ $\parallel \mathbf{G_1-G_2}$	58 $k \xleftarrow{\$} \mathcal{K}$ $\parallel \mathbf{G_3}$
28 if $\mathcal{L}_H[\text{label}, k_1, k_2, c_2, \text{pk}_2] = \perp$	59 $D := D \cup \{(\text{pk}_{2,1-b}, c_1, c_2)\}$ $\parallel \mathbf{G_3}$
29 $k \xleftarrow{\$} \mathcal{K}$	60 $D[\text{pk}_{2,1-b}, c_1, c_2] := k$ $\parallel \mathbf{G_3}$
30 $\mathcal{L}_H := \mathcal{L}_H \cup \{(\text{label}, k_1, k_2, c_2, \text{pk}_2)\}$	61 $k := D[\text{pk}_{2,1-b}, c_1, c_2]$ $\parallel \mathbf{G_3}$
31 $\mathcal{L}_H[\text{label}, k_1, k_2, c_2, \text{pk}_2] := k$	62 return k
32 return $\mathcal{L}_H[\text{label}, k_1, k_2, c_2, \text{pk}_2]$	

Fig. 12. Games $\mathbf{G_0-G_3}$ for the proof of Lemma 2.

Consider the sequence of games $\mathbf{G}_0$ - $\mathbf{G}_3$ shown in Figure 12:

Game $\mathbf{G}_0$: This is the wANO-CCA^* security game for HKEM.

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] = \text{Adv}_{\text{HKEM}}^{\text{wANO-CCA}^*}(\mathcal{A}) + \frac{1}{2}.$$

Game $\mathbf{G}_1$: In this game, for all the hash values $H(\text{label} \| k_1 \| k_2 \| c_2 \| \text{pk}_2)$ which satisfy $\text{Decap}_2(\text{sk}_{2,0}, c_2) = k_2$ (or $\text{Decap}_2(\text{sk}_{2,1}, c_2) = k_2$, resp.), we reprogram them by the corresponding random function value $H'(\text{label} \| k_1 \| c_2 \| \text{pk}_{2,0})$ (or $H'(\text{label} \| k_1 \| c_2 \| \text{pk}_{2,1})$, resp.). By doing this, we do not need the secret key of KEM_2 in the decapsulation oracles. Since there is a bijection between (k_2, c_2, pk_2) and (c_2, pk_2) , this reprogramming does not change the distribution. Therefore,

$$\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] = \Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1].$$

Reduction $\mathcal{B}^{\text{PCO}_2(\cdot, \cdot, \cdot)}(\text{pk}_{2,0}, \text{pk}_{2,1}, c_2^*)$ 01 $(\text{sk}_{1,0}, \text{pk}_{1,0}) \leftarrow \text{KeyGen}_1(\text{par}_1)$ 02 $(\text{sk}_{1,1}, \text{pk}_{1,1}) \leftarrow \text{KeyGen}_1(\text{par}_1)$ 03 $\text{pk}_0 := (\text{pk}_{1,0}, \text{pk}_{2,0})$ 04 $\text{pk}_1 := (\text{pk}_{1,1}, \text{pk}_{2,1})$ 05 $\mathcal{L}_H := \emptyset$ 06 $H' \xleftarrow{\$} \Omega_{H'}$ 07 $(k_1^*, c_1^*) \leftarrow \text{Encap}_1(\text{pk}_{1,1})$ 08 $c^* := (c_1^*, c_2^*)$ 09 $b' \leftarrow \mathcal{A}^{\text{DECAPO}^*(\cdot, \cdot, H(\cdot))}(\text{pk}_0, \text{pk}_1, c^*)$ 10 return b'	Oracle $\text{DECAPO}^*(\text{pk}_0, c)$ 20 parse c as (c_1, c_2) 21 $k_1 := \text{Decap}_1(\text{sk}_{1,0}, c_1)$ 22 $k := H'(\text{label}, k_1, c_2, \text{pk}_{2,0})$ 23 return k Oracle $\text{DECAPO}^*(\text{pk}_1, c)$ 24 parse c as (c_1, c_2) 25 $k_1 := \text{Decap}_1(\text{sk}_{1,1}, c_1)$ 26 $k := H'(\text{label}, k_1, c_2, \text{pk}_{2,1})$ 27 return k
Oracle $H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$ 11 if $\exists \hat{b} \in \{0, 1\}$ s.t. 12 $\text{pk}_2 = \text{pk}_{2,\hat{b}} \wedge \text{PCO}_2(\text{pk}_2, k_2, c_2) = 1$ 13 $\mathcal{L}_H[\text{label}, k_1, k_2, c_2, \text{pk}_2] :=$ 14 $H'(\text{label}, k_1, c_2, \text{pk}_2)$ 15 if $\mathcal{L}_H[\text{label}, k_1, k_2, c_2, \text{pk}_2] = \perp$ 16 $k \xleftarrow{\$} \mathcal{K}$ 17 $\mathcal{L}_H := \mathcal{L}_H \cup \{(\text{label}, k_1, k_2, c_2, \text{pk}_2)\}$ 18 $\mathcal{L}_H[\text{label}, k_1, k_2, c_2, \text{pk}_2] := k$ 19 return $\mathcal{L}_H[\text{label}, k_1, k_2, c_2, \text{pk}_2]$	

Fig. 13. The adversary $\mathcal{B}^{\text{PCO}_2(\cdot, \cdot, \cdot)}$ against the wANO-PCA security of KEM_2 for the proof of Lemma 2. In the reduction, $\mathcal{B}$ queries PCO_2 at most q_H times.

Game $\mathbf{G}_2$: In this game, instead of using the random bit b to get c_2^* , we compute it by fixing the bit to be 0, that is $(k_2^*, c_2^*) \leftarrow \text{Encap}_2(\text{pk}_{2,0})$. Games $\mathbf{G}_2$ and $\mathbf{G}_1$ will only have a difference when $b \xleftarrow{\$} \{0, 1\}$ is chosen to be 1. By using the

reduction shown in Figure 13, we can bound this difference by the wANO-PCA advantage of KEM_2 :

$$\left| \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \right| = \frac{1}{2} \cdot \left| \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1 | b = 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1 | b = 1] \right| \leq \text{Adv}_{\text{KEM}_2}^{\text{wANO-PCA}}(\mathcal{B}).$$

Game $\mathbf{G}_3$: In this game, we use a different way to reprogram the random oracle such that the decapsulation oracles no longer need the secret key of KEM_1 . This can be viewed as a “symmetric” way of the change from $\mathbf{G}_0$ to $\mathbf{G}_1$, but because the input of H does not involve c_1 , we have to record all the queries to the decapsulation oracles and the random oracle in order to keep the consistency. Since $\mathcal{A}$ ’s view is the same as before, we have

$$\Pr[\mathbf{G}_3^{\mathcal{A}} \Rightarrow 1] = \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1].$$

Reduction $\mathcal{C}^{\text{PCO}_1(\cdot, \cdot)}(\text{pk}_{1,0}, \text{pk}_{1,1}, c_1^*)$	Oracle $\text{DECAPO}^*(\text{pk}_0, c)$
01 $(\text{sk}_{2,0}, \text{pk}_{2,0}) \leftarrow \text{KeyGen}_2(\text{par}_2)$	20 parse c as (c_1, c_2)
02 $(\text{sk}_{2,1}, \text{pk}_{2,1}) \leftarrow \text{KeyGen}_2(\text{par}_2)$	21 $k_2 := \text{Decap}_2(\text{sk}_{2,0}, c_2)$
03 $\text{pk}_0 := (\text{pk}_{1,0}, \text{pk}_{2,0})$	22 if $\exists(\text{label}, k_1, k_2, c_2, \text{pk}_{2,0}) \in \mathcal{L}_H$ s.t.
04 $\text{pk}_1 := (\text{pk}_{1,1}, \text{pk}_{2,1})$	23 $\text{PCO}_1(\text{pk}_{1,0}, k_1, c_1) = 1$
05 $\mathcal{L}_H := \emptyset$	24 return $\mathcal{L}_H[\text{label}, k_1, k_2, c_2, \text{pk}_{2,0}]$
06 $\mathbf{D} := \emptyset$	25 else if $\mathbf{D}[\text{pk}_{2,0}, c_1, c_2] = \perp$
07 $(k_2^*, c_2^*) \leftarrow \text{Encap}_2(\text{pk}_{2,0})$	26 $k \xleftarrow{\$} \mathcal{K}$
08 $c^* := (c_1^*, c_2^*)$	27 $\mathbf{D} := \mathbf{D} \cup \{(\text{pk}_{2,0}, c_1, c_2)\}$
09 $b' \leftarrow \mathcal{A}^{\text{DECAPO}^*(\cdot, \cdot), H(\cdot)}(\text{pk}_0, \text{pk}_1, c^*)$	28 $\mathbf{D}[\text{pk}_{2,0}, c_1, c_2] := k$
10 return b'	29 $k := \mathbf{D}[\text{pk}_{2,0}, c_1, c_2]$
Oracle $H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$	30 return k
11 if $\exists(\text{pk}_{2,\hat{b}}, c_1, c_2) \in \mathbf{D}$ s.t. $\text{pk}_2 = \text{pk}_{2,\hat{b}}$	Oracle $\text{DECAPO}^*(\text{pk}_1, c)$
12 if $\text{PCO}_1(\text{pk}_{1,\hat{b}}, k_1, c_1) = 1 \wedge$	31 parse c as (c_1, c_2)
13 $\text{Decap}_2(\text{sk}_{2,\hat{b}}, c_2) = k_2$	32 $k_2 := \text{Decap}_2(\text{sk}_{2,1}, c_2)$
14 return $\mathbf{D}[\text{pk}_{2,\hat{b}}, c_1, c_2]$	33 if $\exists(\text{label}, k_1, k_2, c_2, \text{pk}_{2,1}) \in \mathcal{L}_H$ s.t.
15 if $\mathcal{L}_H[\text{label}, k_1, k_2, c_2, \text{pk}_2] = \perp$	34 $\text{PCO}_1(\text{pk}_{1,1}, k_1, c_1) = 1$
16 $k \xleftarrow{\$} \mathcal{K}$	35 return $\mathcal{L}_H[\text{label}, k_1, k_2, c_2, \text{pk}_{2,1}]$
17 $\mathcal{L}_H := \mathcal{L}_H \cup \{(\text{label}, k_1, k_2, c_2, \text{pk}_2)\}$	36 else if $\mathbf{D}[\text{pk}_{2,1}, c_1, c_2] = \perp$
18 $\mathcal{L}_H[\text{label}, k_1, k_2, c_2, \text{pk}_2] := k$	37 $k \xleftarrow{\$} \mathcal{K}$
19 return $\mathcal{L}_H[\text{label}, k_1, k_2, c_2, \text{pk}_2]$	38 $\mathbf{D} := \mathbf{D} \cup \{(\text{pk}_{2,1}, c_1, c_2)\}$
	39 $\mathbf{D}[\text{pk}_{2,1}, c_1, c_2] := k$
	40 $k := \mathbf{D}[\text{pk}_{2,1}, c_1, c_2]$
	41 return k

Fig. 14. The adversary $\mathcal{C}^{\text{PCO}_1(\cdot, \cdot)}$ against the wANO-PCA security of KEM_1 for the proof of Lemma 2. In the reduction, $\mathcal{C}$ queries PCO_1 at most $O(q_H \cdot q_D)$ times.

Finally, as shown in Figure 14, we can bound the success probability of $\mathcal{A}$ in $\mathbf{G}_3$ by the wANO-PCA advantage of KEM_1 . In the reduction, $\mathcal{C}$ can justify

whether $\text{Decap}_1(\text{sk}_{1,0}, c_1) = k_1$ (or $\text{Decap}_1(\text{sk}_{1,1}, c_1) = k_1$, resp.) by querying the oracle PCO_1 with $(\text{pk}_{1,0}, k_1, c_1)$ (or $(\text{pk}_{1,1}, k_1, c_1)$, resp.), and we have

$$\Pr[\mathbf{G}_3^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_{\text{KEM}_1}^{\text{wANO-PCA}}(\mathcal{C}) + \frac{1}{2}.$$

We have Equation (7) by combining all the bounds above. $\square$

We have shown the wANO-CCA^* security of HKEM in Lemma 2. Since the wANO-CCA^* security is stronger than the wANO-CCA security⁵, we get the ANO-CCA of HKEM by Lemma 1, and we describe it in the following corollary:

Lemma 3 (Anonymity of HKEM (Case II)). *Let $\text{KEM}_1 = (\text{KeyGen}_1, \text{Encap}_1, \text{Decap}_1)$ and $\text{KEM}_2 = (\text{KeyGen}_2, \text{Encap}_2, \text{Decap}_2)$ be two wANO-PCA secure key encapsulation mechanisms. Let $H : \mathcal{L} \times \mathcal{K}_1 \times \mathcal{K}_2 \times \mathcal{C}_2 \times \mathcal{PK}_2 \rightarrow \mathcal{K}$ be a random oracle and let $\mathcal{A}$ be an adversary against the ANO-CCA security of HKEM, making at most q_H queries to the random oracle and q_D queries to the decapsulation oracles. Then there exists an IND-CCA adversary $\mathcal{A}_1$, a wANO-PCA adversary $\mathcal{C}$ against KEM_1 and a wANO-PCA adversary $\mathcal{B}$ against KEM_2 such that*

$$\text{Adv}_{\text{HKEM}}^{\text{ANO-CCA}}(\mathcal{A}) \leq \text{Adv}_{\text{HKEM}}^{\text{IND-CCA}}(\mathcal{A}_1) + \text{Adv}_{\text{KEM}_1}^{\text{wANO-PCA}}(\mathcal{C}) + \text{Adv}_{\text{KEM}_2}^{\text{wANO-PCA}}(\mathcal{B}), \quad (8)$$

and

$$\begin{aligned} \text{Time}(\mathcal{A}_1) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{A}_1) &\approx \text{Mem}(\mathcal{A}), \\ \text{Time}(\mathcal{B}) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{B}) &= \text{Mem}(\mathcal{A}) + \mathcal{O}(q_H), \\ \text{Time}(\mathcal{C}) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{C}) &= \text{Mem}(\mathcal{A}) + \mathcal{O}(q_H) + \mathcal{O}(q_D). \end{aligned}$$

5 Our Variant of X-Wing

In this section, we consider a variant of X-Wing with one change, where our hybrid KEM outputs $K := H(\text{label} \| k_1 \| c_1 \| k_2 \| c_2 \| \text{pk}_2)$ with the additional ciphertext c_1 from KEM_1 . According to the definition in [5], here the label is encoded as 6-byte ASCII string ““\././^””. We denote this variant as HKEM' and prove its ANO-CCA and IND-CCA security in the random oracle model. These changes allow us to give memory-tight reductions. We highlight the changes in Line 09 and Line 16. Let $\text{KEM}_1 = (\text{KeyGen}_1, \text{Encap}_1, \text{Decap}_1)$ and $\text{KEM}_2 = (\text{KeyGen}_2, \text{Encap}_2, \text{Decap}_2)$. The definition of HKEM' is given in Figure 15. Let $\mathcal{K}_1$ and $\mathcal{K}_2$ denote the key space of KEM_1 and KEM_2 respectively, and let $\mathcal{C}_1$ and $\mathcal{C}_2$ denote the ciphertext space of KEM_1 and KEM_2 respectively. Let $\mathcal{PK}_2$ be the public key space of KEM_2 , $\mathcal{L}$ be the label space for label, and $\mathcal{K}$ be the key space for HKEM' .

Theorem 2 shows the wANO-CCA^* security of HKEM' in the ROM, based on the wANO-PCA security of KEM_1 and KEM_2 . By instantiating KEM_1 with ML-KEM-768 and KEM_2 with EG-KEM, EG-KEM satisfies wANO-PCA without

⁵ Because in wANO-CCA^* an adversary can ask decapsulation on a challenge ciphertext.

Algorithm KeyGen(par)	Algorithm Encap(pk)	Algorithm Decap(sk, c)
01 $(sk_1, pk_1) \leftarrow \text{KeyGen}_1(\text{par}_1)$	06 parse pk as (pk_1, pk_2)	12 parse sk as (sk_1, sk_2, pk_2)
02 $(sk_2, pk_2) \leftarrow \text{KeyGen}_2(\text{par}_2)$	07 $(k_1, c_1) \leftarrow \text{Encap}_1(pk_1)$	13 parse c as (c_1, c_2)
03 $pk := (pk_1, pk_2)$	08 $(k_2, c_2) \leftarrow \text{Encap}_2(pk_2)$	14 $k_1 := \text{Decap}_1(sk_1, c_1)$
04 $sk := (sk_1, sk_2)$	09 $k := H(\text{label} \ k_1 \ c_1 \ k_2 \ c_2 \ pk_2)$	15 $k_2 := \text{Decap}_2(sk_2, c_2)$
05 return (pk, sk)	10 $c := (c_1, c_2)$	16 $k := H(\text{label} \ k_1 \ c_1 \ k_2 \ c_2 \ pk_2)$
	11 return (k, c)	17 return k

Fig. 15. Definition of the variant HKEM'.

any assumption (cf. Lemma 4). Thus we can conclude that HKEM' is wANO-CCA* (and also wANO-CCA) if ML-KEM-768 is wANO-PCA secure (cf. Theorem 4).

Then, we analyze the IND-CCA security of HKEM', as shown in Theorem 3. The IND-CCA security of HKEM' only requires the OW-PCA security of either KEM₁ or KEM₂ in the ROM. By combining Lemma 1 and Theorem 2, we obtain the ANO-CCA security of HKEM', with the tightness of advantage, time, and memory.

To summarize, if we instantiate KEM₁ and KEM₂ with ML-KEM-768 and EG-KEM respectively, then we can obtain the ANO-CCA security of HKEM' under the wANO-PCA and OW-PCA security of ML-KEM-768, without any other assumption. Or, we could also reduce it to the wANO-PCA security of ML-KEM-768 and the StCDH assumption, where the latter implies the OW-PCA security of EG-KEM.

The following Theorem 2 gives a memory-tight reduction from wANO-CCA* of HKEM' to wANO-PCA of both KEM₁ and KEM₂. Comparing to Lemma 2, this reduction can achieve the memory tightness because c_1 is included in the hash query. As pointed out in [5], when implementing H with SHA3-256, omitting c_1 (which is the ciphertext of ML-KEM-768) from the input of H provides about a 9% performance improvement. Hence, we trade this 9% efficiency for memory-tightness, subsequently, stronger security guarantees. We mainly view our HKEM' as a theoretical contribution, rather than a drop-in practical replacement.

Theorem 2 (wANO-CCA* of HKEM'). *Let $\text{KEM}_1 = (\text{KeyGen}_1, \text{Encap}_1, \text{Decap}_1)$ and $\text{KEM}_2 = (\text{KeyGen}_2, \text{Encap}_2, \text{Decap}_2)$ be two wANO-PCA secure key encapsulation mechanisms. Let $H : \mathcal{L} \times \mathcal{K}_1 \times \mathcal{C}_1 \times \mathcal{K}_2 \times \mathcal{C}_2 \times \mathcal{PK}_2 \rightarrow \mathcal{K}$ be a random oracle and let $\mathcal{A}$ be an adversary against the wANO-CCA* security of the hybrid KEM HKEM', making at most q_H queries to the random oracle and q_D queries to the decryption oracle. Then there exists a wANO-PCA adversary $\mathcal{B}$ against KEM₂ and a wANO-PCA adversary $\mathcal{C}$ against KEM₁ such that*

$$\text{Adv}_{\text{HKEM}'}^{\text{wANO-CCA}^*}(\mathcal{A}) \leq \text{Adv}_{\text{KEM}_1}^{\text{wANO-PCA}}(\mathcal{C}) + \text{Adv}_{\text{KEM}_2}^{\text{wANO-PCA}}(\mathcal{B}) + \text{Adv}_{\text{KEM}_2}^{\text{COLL}}, \quad (9)$$

and

$$\begin{aligned} \text{Time}(\mathcal{B}) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{B}) &\approx \text{Mem}(\mathcal{A}), \\ \text{Time}(\mathcal{C}) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{C}) &\approx \text{Mem}(\mathcal{A}). \end{aligned}$$

PROOF. Let $\mathcal{A}$ be an adversary against the wANO-CCA* security of HKEM'. Consider the sequence of games $\mathbf{G}_0$ - $\mathbf{G}_2$ shown in Figure 16.

Game $\mathbf{G}_0 - \mathbf{G}_2$	Oracle $\text{DECAPO}^*(\text{pk}_0, c)$
01 $(\text{sk}_{1,0}, \text{pk}_{1,0}) \leftarrow \text{KeyGen}_1(\text{par}_1)$	21 parse c as (c_1, c_2)
02 $(\text{sk}_{1,0}, \text{pk}_{1,1}) \leftarrow \text{KeyGen}_1(\text{par}_1)$	22 $k_1 := \text{Decap}_1(\text{sk}_{1,0}, c_1)$
03 $(\text{sk}_{2,0}, \text{pk}_{2,0}) \leftarrow \text{KeyGen}_2(\text{par}_2)$	23 $k_2 := \text{Decap}_2(\text{sk}_{2,0}, c_2)$
04 $(\text{sk}_{2,1}, \text{pk}_{2,1}) \leftarrow \text{KeyGen}_2(\text{par}_2)$	24 $k = H(\text{label} \ k_1 \ c_1 \ k_2 \ c_2 \ \text{pk}_{2,0})$ $\parallel \mathbf{G}_0$
05 if $\text{pk}_{2,0} = \text{pk}_{2,1}$ $\parallel \mathbf{G}_1 - \mathbf{G}_2$	25 $k = H_0(\text{label} \ \star \ c_1 \ \star \ c_2 \ \text{pk}_{2,0})$ $\parallel \mathbf{G}_1 - \mathbf{G}_2$
06 BAD := true; abort $\parallel \mathbf{G}_1 - \mathbf{G}_2$	26 return k
07 $\text{pk}_0 := (\text{pk}_{1,0}, \text{pk}_{2,0})$	Oracle $\text{DECAPO}^*(\text{pk}_1, c)$
08 $\text{pk}_1 := (\text{pk}_{1,1}, \text{pk}_{2,1})$	27 parse c as (c_1, c_2)
09 $\text{sk}_0 := (\text{sk}_{1,0}, \text{sk}_{2,0})$	28 $k_1 := \text{Decap}_1(\text{sk}_{1,1}, c_1)$
10 $\text{sk}_1 := (\text{sk}_{2,1}, \text{sk}_{2,1})$	29 $k_2 := \text{Decap}_2(\text{sk}_{2,1}, c_2)$
11 $b \xleftarrow{\$} \{0, 1\}$	30 $k = H(\text{label} \ k_1 \ c_1 \ k_2 \ c_2 \ \text{pk}_{2,1})$ $\parallel \mathbf{G}_0$
12 $H_0, H_1 \xleftarrow{\$} \Omega_H$ $\parallel \mathbf{G}_1 - \mathbf{G}_2$	31 $k = H_1(\text{label} \ \star \ c_1 \ \star \ c_2 \ \text{pk}_{2,1})$ $\parallel \mathbf{G}_1 - \mathbf{G}_2$
13 $(c_1^*, k_1^*) \leftarrow \text{Encap}_1(\text{pk}_{1,b})$	32 return k
14 $(c_2^*, k_2^*) \leftarrow \text{Encap}_2(\text{pk}_{2,b})$ $\parallel \mathbf{G}_0 - \mathbf{G}_1$	Oracle $H(\text{label} \ k_1 \ c_1 \ k_2 \ c_2 \ \text{pk}_2)$
15 $(c_2^*, k_2^*) \leftarrow \text{Encap}_2(\text{pk}_{2,0})$ $\parallel \mathbf{G}_2$	33 if $\exists \hat{b}$ s.t. $\text{pk}_{2,\hat{b}} = \text{pk}_2$ $\parallel \mathbf{G}_1 - \mathbf{G}_2$
16 $K^* := H(\text{label} \ k_1^* \ c_1^* \ k_2^* \ c_2^* \ \text{pk}_{2,b})$ $\parallel \mathbf{G}_0$	34 if $\text{Decap}_1(\text{sk}_{1,\hat{b}}, c_1) = k_1 \wedge$
17 $K^* := H_b(\text{label} \ \star \ c_1^* \ \star \ c_2^* \ \text{pk}_{2,b})$ $\parallel \mathbf{G}_1 - \mathbf{G}_2$	35 $\text{Decap}_2(\text{sk}_{2,\hat{b}}, c_2) = k_2$ $\parallel \mathbf{G}_1 - \mathbf{G}_2$
18 $c^* := (c_1^*, c_2^*)$	36 return $H_{\hat{b}}(\text{label} \ \star \ c_1 \ \star \ c_2 \ \text{pk}_2)$ $\parallel \mathbf{G}_1 - \mathbf{G}_2$
19 $b' \leftarrow \mathcal{A}^{\text{DECAPO}^*(\cdot, \cdot), H(\cdot)}(\text{pk}_0, \text{pk}_1, c^*)$	37 return $H(\text{label} \ k_1 \ c_1 \ k_2 \ c_2 \ \text{pk}_2)$
20 return $\llbracket b = b' \rrbracket$	

Fig. 16. Games $\mathbf{G}_0 - \mathbf{G}_2$ for the proof of Theorem 2.

Game $\mathbf{G}_0$: This is the wANO-CCA^* security game for HKEM' .

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] = \text{Adv}_{\text{HKEM}'}^{\text{wANO-CCA}^*}(\mathcal{A}) + \frac{1}{2}.$$

Game $\mathbf{G}_1$: In this game, we change all the possible decapsulation values of HKEM' by reprogramming the random oracle H (i.e. including all the hash values that might be used in the decapsulation oracles and the value of $K^* = H(\text{label} \| k_1^* \| c_1^* \| k_2^* \| c_2^* \| \text{pk}_{2,b})$). Specifically speaking, for all the tuples $(\text{label} \| k_1 \| c_1 \| k_2 \| c_2 \| \text{pk}_2)$ satisfying $\text{pk}_{2,\hat{b}} = \text{pk}_2$ (here $\hat{b} \in \{0, 1\}$ denotes which public key of KEM_2 corresponds to pk_2), if $k_1 = \text{Decap}_1(\text{sk}_{1,\hat{b}}, c_1) \wedge k_2 = \text{Decap}_2(\text{sk}_{2,\hat{b}}, c_2)$, we change the corresponding hash value into $H_{\hat{b}}(\text{label} \| \star \| c_1 \| \star \| c_2 \| \text{pk}_2)$.

Define a event $\text{BAD} : \text{pk}_{2,0} = \text{pk}_{2,1}$, then $\mathcal{A}$'s view in $\mathbf{G}_0$ and $\mathbf{G}_1$ are the same when BAD never occurs. Since the probability of BAD happening can be bounded by the collision advantage of public keys of KEM_2 , we have

$$\left| \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] \right| \leq \Pr[\text{BAD}] \leq \text{Adv}_{\text{KEM}_2}^{\text{COLL}}.$$

Game $\mathbf{G}_2$: In this game, instead of using the random bit b to get c_2^* , we compute it by fixing the bit to be 0, that is $(c_2^*, k_2^*) \leftarrow \text{Encap}_2(\text{pk}_{2,0})$. Games $\mathbf{G}_2$ and $\mathbf{G}_1$ will only have a difference when $b \xleftarrow{\$} \{0, 1\}$ is chosen to be 1. By using the reduction shown in Figure 17, we can bound this difference by the wANO-PCA advantage of KEM_2 :

$$\left| \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \right| = \frac{1}{2} \cdot \left| \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1 | b = 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1 | b = 1] \right|$$

Reduction $\mathcal{B}_2^{\text{PCO}_2(\cdot, \cdot, \cdot)}(\text{pk}_{2,0}, \text{pk}_{2,1}, c_2^*)$	Oracle $H(\text{label} \ k_1 \ c_1 \ k_2 \ c_2 \ \text{pk}_2)$
01 $(\text{sk}_{1,0}, \text{pk}_{1,0}) \leftarrow \text{KeyGen}_1(\text{par}_1)$	13 if $\exists \hat{b}$ s.t. $\text{pk}_{2,\hat{b}} = \text{pk}_2$
02 $(\text{sk}_{1,0}, \text{pk}_{1,1}) \leftarrow \text{KeyGen}_1(\text{par}_1)$	14 if $\text{Decap}_1(\text{sk}_{1,\hat{b}}, c_1) = k_1 \wedge$
03 $\text{pk}_0 := (\text{pk}_{1,0}, \text{pk}_{2,0})$	15 $\text{PCO}_2(\text{pk}_{2,\hat{b}}, k_2, c_2) = 1$
04 $\text{pk}_1 := (\text{pk}_{1,1}, \text{pk}_{2,1})$	16 return $H_{\hat{b}}(\text{label} \ \star \ c_1 \ \star \ c_2 \ \text{pk}_2)$
05 $(c_1^*, k_1^*) \leftarrow \text{Encap}_1(\text{pk}_{1,1})$	17 return $H(\text{label} \ k_1 \ c_1 \ k_2 \ c_2 \ \text{pk}_2)$
06 $c^* := (c_1^*, c_2^*)$	
07 $H_0, H_1 \xleftarrow{\$} \Omega_H$	Oracle $\text{DECAPO}^*(\text{pk}_1, c)$
08 $b' \leftarrow \mathcal{A}^{\text{DECAPO}^*(\cdot, \cdot), H(\cdot)}(\text{pk}_0, \text{pk}_1, c^*)$	18 parse c as (c_1, c_2)
09 return b'	19 $k = H_1(\text{label} \ \star \ c_1 \ \star \ c_2 \ \text{pk}_{2,1})$
	20 return k
Oracle $\text{DECAPO}^*(\text{pk}_0, c)$	
10 parse c as (c_1, c_2)	
11 $k = H_0(\text{label} \ \star \ c_1 \ \star \ c_2 \ \text{pk}_{2,0})$	
12 return k	

Fig. 17. The adversary $\mathcal{B}_2^{\text{PCO}_2(\cdot, \cdot, \cdot)}$ against the wANO-PCA security of KEM_2 for the proof of Theorem 2. In the reduction, $\mathcal{B}_2$ queries the PCO_2 oracle at most $2q_H$ times.

Reduction $\mathcal{C}^{\text{PCO}_1(\cdot, \cdot, \cdot)}(\text{pk}_{1,0}, \text{pk}_{1,1}, c_1^*)$	Oracle $H(\text{label} \ k_1 \ c_1 \ k_2 \ c_2 \ \text{pk}_2)$
01 $(\text{sk}_{2,0}, \text{pk}_{2,0}) \leftarrow \text{KeyGen}_2(\text{par}_2)$	13 if $\exists \hat{b}$ s.t. $\text{pk}_{2,\hat{b}} = \text{pk}_2$
02 $(\text{sk}_{2,0}, \text{pk}_{2,1}) \leftarrow \text{KeyGen}_2(\text{par}_2)$	14 if $\text{PCO}_1(\text{pk}_{1,\hat{b}}, k_1, c_1) = 1 \wedge$
03 $\text{pk}_0 := (\text{pk}_{1,0}, \text{pk}_{2,0})$	15 $\text{Decap}_2(\text{sk}_{2,\hat{b}}, c_2) = k_2$
04 $\text{pk}_1 := (\text{pk}_{1,1}, \text{pk}_{2,1})$	16 return $H_{\hat{b}}(\text{label} \ \star \ c_1 \ \star \ c_2 \ \text{pk}_2)$
05 $(c_2^*, k_2^*) \leftarrow \text{Encap}_2(\text{pk}_{2,0})$	17 return $H(\text{label} \ k_1 \ c_1 \ k_2 \ c_2 \ \text{pk}_2)$
06 $c^* := (c_1^*, c_2^*)$	
07 $H_0, H_1 \xleftarrow{\$} \Omega_H$	Oracle $\text{DECAPO}^*(\text{pk}_1, c)$
08 $b' \leftarrow \mathcal{A}^{\text{DECAPO}^*(\cdot, \cdot), H(\cdot)}(\text{pk}_0, \text{pk}_1, c^*)$	18 parse c as (c_1, c_2)
09 return b'	19 $k = H_1(\text{label} \ \star \ c_1 \ \star \ c_2 \ \text{pk}_{2,1})$
	20 return k
Oracle $\text{DECAPO}^*(\text{pk}_0, c)$	
10 parse c as (c_1, c_2)	
11 $k = H_0(\text{label} \ \star \ c_1 \ \star \ c_2 \ \text{pk}_{2,0})$	
12 return k	

Fig. 18. The adversary $\mathcal{C}^{\text{PCO}_1(\cdot, \cdot, \cdot)}$ against the wANO-PCA security of KEM_1 for the proof of Theorem 2. In the reduction, $\mathcal{C}$ queries the PCO_1 oracle at most $2q_H$ times.

$$\leq \text{Adv}_{\text{KEM}_2}^{\text{wANO-PCA}}(\mathcal{B}_2).$$

Finally, as shown in Figure 18, we can bound the success probability of $\mathcal{A}$ in $\mathbf{G}_2$ by the wANO-PCA advantage of KEM_1 , therefore

$$\Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_{\text{KEM}_1}^{\text{wANO-PCA}}(\mathcal{C}) + \frac{1}{2}.$$

We have Equation (9) by combining all the bounds above. $\square$

As shown in Lemma 1, wANO-CCA and IND-CCA together implies ANO-CCA, so we prove the IND-CCA security of HKEM' in the following Theorem 3. Our proof shows that IND-CCA of HKEM' only requires one of the underlying KEM is OW-PCA, and the reduction is memory-tight.

Theorem 3 (OW-PCA₁/OW-PCA₂ $\Rightarrow$ IND-CCA). *Let $\text{KEM}_1 = (\text{KeyGen}_1, \text{Encap}_1, \text{Decap}_1)$ and $\text{KEM}_2 = (\text{KeyGen}_2, \text{Encap}_2, \text{Decap}_2)$ be two key encapsulation mechanisms. Let $H : \mathcal{L} \times \mathcal{K}_1 \times \mathcal{C}_1 \times \mathcal{K}_2 \times \mathcal{C}_2 \times \mathcal{PK}_2 \rightarrow \mathcal{K}$ be a random oracle and let $\mathcal{A}$ be an adversary against the IND-CCA security of the hybrid KEM HKEM', making at most q_H queries to the random oracle and q_D queries to the decryption oracle. Then there exists an OW-PCA adversary $\mathcal{B}$ against KEM_1 or KEM_2 such that*

$$\text{Adv}_{\text{HKEM}'}^{\text{IND-CCA}}(\mathcal{A}) \leq \min(\text{Adv}_{\text{KEM}_1}^{\text{OW-PCA}}(\mathcal{B}), \text{Adv}_{\text{KEM}_2}^{\text{OW-PCA}}(\mathcal{B})), \quad (10)$$

and

$$\text{Time}(\mathcal{B}) \approx \text{Time}(\mathcal{A}), \quad \text{Mem}^*(\mathcal{B}) \approx \text{Mem}(\mathcal{A}).$$

PROOF. Let $\mathcal{A}$ be an adversary against the IND-CCA security of HKEM'. Consider the sequence of games $\mathbf{G}_0$ - $\mathbf{G}_2$ shown in Figure 19.

<u>Game $\mathbf{G}_0 - \mathbf{G}_2$</u>	<u>Oracle $H(\text{label} \ k_1 \ c_1 \ k_2 \ c_2 \ \text{pk}_2)$</u>
01 $(\text{sk}_1, \text{pk}_1) \leftarrow \text{KeyGen}_1(\text{par}_1)$	16 if $\text{Decap}_1(\text{sk}_1, c_1) = k_1 \wedge$
02 $(\text{sk}_2, \text{pk}_2) \leftarrow \text{KeyGen}_2(\text{par}_2)$	17 $\text{Decap}_2(\text{sk}_2, c_2) = k_2 \quad \parallel \mathbf{G}_1\text{-}\mathbf{G}_2$
03 $\text{pk} := (\text{pk}_1, \text{pk}_2)$	18 if $c_1 = c_1^* \wedge c_2 = c_2^* \quad \parallel \mathbf{G}_2$
04 $\text{sk} := (\text{sk}_1, \text{sk}_2)$	19 BAD := true ; abort $\parallel \mathbf{G}_2$
05 $(c_1^*, k_1^*) \leftarrow \text{Encap}_1(\text{pk}_1)$	20 return $H'(\text{label} \ * \ c_1 \ * \ c_2 \ \text{pk}_2) \parallel \mathbf{G}_1\text{-}\mathbf{G}_2$
06 $(c_2^*, k_2^*) \leftarrow \text{Encap}_2(\text{pk}_2)$	21 return $H(\text{label} \ k_1 \ c_1 \ k_2 \ c_2 \ \text{pk}_2)$
07 $c^* := (c_1^*, c_2^*)$	<u>Oracle $\text{DECAPO}(\text{pk}, c)$</u>
08 $K_0 \xleftarrow{\$} \mathcal{K}$	22 if $c = c^*$
09 $K_1 := H(\text{label} \ k_1^* \ c_1^* \ k_2^* \ c_2^* \ \text{pk}_2) \parallel \mathbf{G}_0$	23 return $\perp$
10 $H' \xleftarrow{\$} \Omega_H$	24 parse c as (c_1, c_2)
11 $K_1 := H'(\text{label} \ * \ c_1^* \ * \ c_2^* \ \text{pk}_2) \parallel \mathbf{G}_1\text{-}\mathbf{G}_2$	25 $k_1 := \text{Decap}_1(\text{sk}_1, c_1) \parallel \mathbf{G}_0$
12 $K_1 \xleftarrow{\$} \mathcal{K} \parallel \mathbf{G}_2$	26 $k_2 := \text{Decap}_2(\text{sk}_2, c_2) \parallel \mathbf{G}_0$
13 $b \xleftarrow{\$} \{0, 1\}$	27 $k := H(\text{label} \ k_1 \ c_1 \ k_2 \ c_2 \ \text{pk}_2) \parallel \mathbf{G}_0$
14 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\text{pk}, \cdot), H(\cdot)}(\text{pk}, (c^*, K_b^*))$	28 $k = H'(\text{label} \ * \ c_1 \ * \ c_2 \ \text{pk}_2) \parallel \mathbf{G}_1\text{-}\mathbf{G}_2$
15 return $\llbracket b = b' \rrbracket$	29 return k

Fig. 19. Games $\mathbf{G}_0$ - $\mathbf{G}_2$ for the proof of Theorem 3.

Game $\mathbf{G}_0$: This is the IND-CCA security game for HKEM'.

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] = \text{Adv}_{\text{HKEM}'}^{\text{IND-CCA}}(\mathcal{A}).$$

Game $\mathbf{G}_1$: In this game, we reprogram the hash values of all the tuples $(\text{label} \| k_1 \| c_1 \| k_2 \| c_2 \| \text{pk}_2)$ which satisfy $k_1 = \text{Decap}(\text{sk}_1, c_1) \wedge k_2 = \text{Decap}(\text{sk}_2, c_2)$. Specifically, if $k_1 = \text{Decap}(\text{sk}_1, c_1) \wedge k_2 = \text{Decap}(\text{sk}_2, c_2)$, then we replace the corresponding hash value of $(\text{label} \| k_1 \| c_1 \| k_2 \| c_2 \| \text{pk}_2)$ with the value of $H'(\text{label} \| * \| c_1 \| * \| c_2 \| \text{pk}_2)$,

where H' is a random function. By doing this, we no longer need the secret key in the decapsulation oracle.

Since there is a bijection between $(\text{Decap}(\text{sk}_1, c_1), c_1, \text{Decap}(\text{sk}_2, c_2), c_2)$ and $(\star, c_1, \star, c_2)$, this change preserves the distribution, we have

$$\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] = \Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1].$$

Game $\mathbf{G}_2$: The only difference between $\mathbf{G}_2$ and $\mathbf{G}_1$ is that we choose k uniformly random here. If the adversary $\mathcal{A}$ never queries the random oracle H with $(\text{label} \| k_1^* \| c_1^* \| k_2^* \| c_2^* \| \text{pk}_2)$, then the environment of these two games are exactly the same. We denote this event as **BAD** and can bound the difference of $\mathbf{G}_1$ and $\mathbf{G}_2$ by it.

Without loss of generality, here we reduce the probability of **BAD** to the OW-PCA security of KEM_1 . As described in Figure 20, if **BAD** occurs, then $\mathcal{B}$ can win the OW-PCA game of KEM_1 by returning the corresponding k_1 . Thus we have

$$\left| \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \right| \leq \Pr[\text{BAD}] \leq \text{Adv}_{\text{KEM}_1}^{\text{OW-PCA}}(\mathcal{B}).$$

If KEM_1 is not OW-PCA secure, we can also reduce the probability of **BAD** to the OW-PCA security of KEM_2 by using a similar reduction, where we only need to substitute KEM_1 and KEM_2 . Therefore,

$$\left| \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \right| \leq \Pr[\text{BAD}] \leq \min(\text{Adv}_{\text{KEM}_1}^{\text{OW-PCA}}(\mathcal{B}), \text{Adv}_{\text{KEM}_2}^{\text{OW-PCA}}(\mathcal{B})).$$

Reduction $\mathcal{B}^{\text{PCO}_1(\text{pk}_1, \cdot, \cdot)}(\text{pk}_1, c_1^*)$	Oracle $H(\text{label} \ k_1 \ c_1 \ k_2 \ c_2 \ \text{pk}_2)$
01 $(\text{sk}_2, \text{pk}_2) \leftarrow \text{KeyGen}_2(\text{par}_2)$	09 if $\text{PCO}_1(\text{pk}_1, k_1, c_1) = 1$
02 $\text{pk} := (\text{pk}_1, \text{pk}_2)$	10 if $c_1 = c_1^*$
03 $(c_2^*, k_2^*) \leftarrow \text{Encap}_2(\text{pk}_2)$	11 BAD $:= \text{true}; \text{abort}$
04 $c^* := (c_1^*, c_2^*)$	12 else if $\text{Decap}_2(\text{sk}_2, c_2) = k_2$
05 $K^* \xleftarrow{\$} \mathcal{K}$	13 return $H'(\text{label} \ \star \ c_1 \ \star \ c_2 \ \text{pk}_2)$
06 $H' \xleftarrow{\$} \Omega_H$	14 return $H(\text{label} \ k_1 \ c_1 \ k_2 \ c_2 \ \text{pk}_2)$
07 $b' \leftarrow \mathcal{A}^{\text{DECAPO}(\text{pk}, \cdot), H(\cdot)}(\text{pk}, (c^*, K^*))$	Oracle $\text{DECAPO}(\text{pk}, c)$
08 abort	15 if $c = c^*$
	16 return $\perp$
	17 parse c as (c_1, c_2)
	18 $k = H'(\text{label} \ \star \ c_1 \ \star \ c_2 \ \text{pk}_2)$
	19 return k

Fig. 20. The adversary $\mathcal{B}^{\text{PCO}_1(\text{pk}_1, \cdot, \cdot)}$ against the OW-PCA security of HKEM' for the proof of Theorem 3. In the reduction, $\mathcal{B}$ queries the PCO_1 oracle at most q_H times.

Since in this game, both K_0 and K_1 are chosen uniformly random, we have

$$\Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] = \frac{1}{2}.$$

We have Equation (10) by combining all the bounds above. $\square$

Acknowledgments. We thank the anonymous reviewers from PKC 2026 and Taehun Kang for their careful reading of our paper and valuable suggestions for a better comparison with the work of Günther et al.[24] and a more detailed efficiency comparison between our HKEM' and the original X-Wing scheme.

References

1. Albrecht, M.R., Bernstein, D.J., Chou, T., Cid, C., Gilcher, J., Lange, T., Maram, V., von Maurich, I., Misoczki, R., Niederhagen, R., Paterson, K.G., Persichetti, E., Peters, C., Schwabe, P., Sendrier, N., Szefer, J., Tjhai, C.J., Tomlinson, M., Wang, W.: Classic McEliece. Tech. rep., National Institute of Standards and Technology (2020), available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions> 5
2. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology* 9(3), 169–203 (2015), <https://doi.org/10.1515/jmc-2015-0016> 3
3. Alwen, J., Blanchet, B., Hauck, E., Kiltz, E., Lipp, B., Riepel, D.: Analysing the HPKE standard. In: Canteaut, A., Standaert, F.X. (eds.) EUROCRYPT 2021, Part I. LNCS, vol. 12696, pp. 87–116. Springer, Cham (Oct 2021) 31
4. Auerbach, B., Cash, D., Fersch, M., Kiltz, E.: Memory-tight reductions. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017, Part I. LNCS, vol. 10401, pp. 101–132. Springer, Cham (Aug 2017) 3, 12
5. Barbosa, M., Connolly, D., Duarte, J.D., Kaiser, A., Schwabe, P., Varner, K., Westerbaan, B.: X-Wing. *CiC* 1(1), 21 (2024) 2, 4, 5, 11, 12, 22, 23, 31, 36, 38, 39, 40
6. Becker, A., Ducas, L., Gama, N., Laarhoven, T.: New directions in nearest neighbor searching with applications to lattice sieving. In: Krauthgamer, R. (ed.) 27th SODA. pp. 10–24. ACM-SIAM (Jan 2016) 3
7. Beguinet, H., Chevalier, C., Pointcheval, D., Ricosset, T., Rossi, M.: GeT a CAKE: Generic transformations from key encapsulation mechanisms to password authenticated key exchanges. In: Tibouchi, M., Wang, X. (eds.) ACNS 2023, Part II. LNCS, vol. 13906, pp. 516–538. Springer, Cham (Jun 2023) 1, 2
8. Bellare, M., Boldyreva, A., Micali, S.: Public-key encryption in a multi-user setting: Security proofs and improvements. In: Preneel, B. (ed.) EUROCRYPT 2000. LNCS, vol. 1807, pp. 259–274. Springer, Berlin, Heidelberg (May 2000) 3
9. Bellare, M., Ristenpart, T.: Simulation without the artificial abort: Simplified proof and improved concrete security for Waters' IBE scheme. In: Joux, A. (ed.) EUROCRYPT 2009. LNCS, vol. 5479, pp. 407–424. Springer, Berlin, Heidelberg (Apr 2009) 3
10. Bellare, M., Rogaway, P.: The exact security of digital signatures: How to sign with RSA and Rabin. In: Maurer, U.M. (ed.) EUROCRYPT'96. LNCS, vol. 1070, pp. 399–416. Springer, Berlin, Heidelberg (May 1996) 3
11. Ben-Sasson, E., Chiesa, A., Garman, C., Green, M., Miers, I., Tromer, E., Virza, M.: Zerocash: Decentralized anonymous payments from bitcoin. In: 2014 IEEE Symposium on Security and Privacy. pp. 459–474. IEEE Computer Society Press (May 2014) 2

12. Bernstein, D.J., Bhargavan, K., Bhasin, S., Chattopadhyay, A., Chia, T.K., Kan-wischer, M.J., Kiefer, F., Paiva, T., Ravi, P., Tamvada, G.: KyberSlash: Exploiting secret-dependent division timings in kyber implementations. TCHES 2025 (2025), <https://eprint.iacr.org/2024/1049> 2
13. Bhattacharyya, R.: Memory-tight reductions for practical key encapsulation mechanisms. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) PKC 2020, Part I. LNCS, vol. 12110, pp. 249–278. Springer, Cham (May 2020) 12
14. Camenisch, J., Lysyanskaya, A.: An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In: Pfitzmann, B. (ed.) EUROCRYPT 2001. LNCS, vol. 2045, pp. 93–118. Springer, Berlin, Heidelberg (May 2001) 2
15. Chen, Y., Nguyen, P.Q.: BKZ 2.0: Better lattice security estimates. In: Lee, D.H., Wang, X. (eds.) ASIACRYPT 2011. LNCS, vol. 7073, pp. 1–20. Springer, Berlin, Heidelberg (Dec 2011) 3
16. Connolly, C., Wood, C.A., Ounsworth, M., van der Merwe, T., Fluhrer, S., McGrew, D.: X-wing: General-purpose hybrid post-quantum kem. Internet-Draft draft-connolly-cfrg-xwing-kem-09, Internet Engineering Task Force (September 2025), <https://datatracker.ietf.org/doc/draft-connolly-cfrg-xwing-kem/>, work in Progress 2
17. D’Anvers, J.P., Karmakar, A., Roy, S.S., Vercauteren, F.: Saber: Module-LWR based key exchange, CPA-secure encryption and CCA-secure KEM. In: Joux, A., Nitaj, A., Rachidi, T. (eds.) AFRICACRYPT 18. LNCS, vol. 10831, pp. 282–305. Springer, Cham (May 2018) 5
18. Fujisaki, E., Okamoto, T.: How to enhance the security of public-key encryption at minimum cost. In: Imai, H., Zheng, Y. (eds.) PKC’99. LNCS, vol. 1560, pp. 53–68. Springer, Berlin, Heidelberg (Mar 1999) 4
19. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. In: Wiener, M.J. (ed.) CRYPTO’99. LNCS, vol. 1666, pp. 537–554. Springer, Berlin, Heidelberg (Aug 1999) 4, 7
20. Fujisaki, E., Okamoto, T.: Secure integration of asymmetric and symmetric encryption schemes. Journal of Cryptology 26(1), 80–101 (Jan 2013) 4
21. Ghoshal, A., Ghosal, R., Jaeger, J., Tessaro, S.: Hiding in plain sight: Memory-tight proofs via randomness programming. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part II. LNCS, vol. 13276, pp. 706–735. Springer, Cham (May / Jun 2022) 5, 12
22. Giacon, F., Heuer, F., Poettering, B.: KEM combiners. In: Abdalla, M., Dahab, R. (eds.) PKC 2018, Part I. LNCS, vol. 10769, pp. 190–218. Springer, Cham (Mar 2018) 11
23. Grubbs, P., Maram, V., Paterson, K.G.: Anonymous, robust post-quantum public key encryption. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part III. LNCS, vol. 13277, pp. 402–432. Springer, Cham (May / Jun 2022) 4, 6, 8
24. Günther, F., Rosenberg, M., Stebila, D., Veitch, S.: Hybrid obfuscated key exchange and KEMs. In: Kalai, Y.T., Kamara, S.F. (eds.) CRYPTO 2025, Part III. LNCS, vol. 16002, pp. 575–609. Springer, Cham (Aug 2025) 4, 11, 12, 18, 28
25. Herold, G., Kirshanova, E.: Improved algorithms for the approximate k -list problem in euclidean norm. In: Fehr, S. (ed.) PKC 2017, Part I. LNCS, vol. 10174, pp. 16–40. Springer, Berlin, Heidelberg (Mar 2017) 3
26. Herold, G., Kirshanova, E., May, A.: On the asymptotic complexity of solving LWE. DCC 86(1), 55–83 (2018) 3

27. Hofheinz, D., Hövelmanns, K., Kiltz, E.: A modular analysis of the Fujisaki-Okamoto transformation. In: Kalai, Y., Reyzin, L. (eds.) TCC 2017, Part I. LNCS, vol. 10677, pp. 341–371. Springer, Cham (Nov 2017) [4](#), [7](#)
28. Hövelmanns, K., Kiltz, E., Schäge, S., Unruh, D.: Generic authenticated key exchange in the quantum random oracle model. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) PKC 2020, Part II. LNCS, vol. 12111, pp. 389–422. Springer, Cham (May 2020) [1](#)
29. Katz, J., Lindell, Y.: Introduction to Modern Cryptography. CRC Press, Boca Raton, FL, 3rd edn. (2021), see Section 12.3, Hybrid Encryption: The KEM/DEM Framework [1](#)
30. Langley, A., Hamburg, M., Turner, S.: Elliptic Curves for Security. RFC 7748 (Jan 2016), <https://www.rfc-editor.org/info/rfc7748> [2](#), [31](#)
31. Lyubashevsky, V., Ducas, L., Kiltz, E., Lepoint, T., Schwabe, P., Seiler, G., Stehlé, D., Bai, S.: CRYSTALS-DILITHIUM. Tech. rep., National Institute of Standards and Technology (2020), available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions> [1](#)
32. Naehrig, M., Alkim, E., Bos, J., Ducas, L., Easterbrook, K., LaMacchia, B., Longa, P., Mironov, I., Nikolaenko, V., Peikert, C., Raghunathan, A., Stebila, D.: FrodoKEM. Tech. rep., National Institute of Standards and Technology (2020), available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions> [5](#)
33. Ni, Z., Khalid, A., Liu, W., O’Neill, M.: Bitstream fault injection attacks on crystals kyber implementations on fpgas. In: 2024 Design, Automation & Test in Europe Conference & Exhibition (DATE). pp. 1–6 (2024) [2](#)
34. NIST: Module-lattice-based key-encapsulation mechanism standard (Aug 2024), <https://doi.org/10.6028/NIST.FIPS.203> [2](#), [32](#)
35. Pablos, J.I.E., González Vasco, M.I., Pérez del Pozo, A.L., Soriente, C.: Anonymous authenticated key exchange. In: Fischlin, M., Moonsamy, V. (eds.) ACNS 2025, Part I. LNCS, vol. 15825, pp. 516–546. Springer, Cham (Jun 2025) [2](#)
36. Pan, J., Wagner, B., Zeng, R.: Tighter security for generic authenticated key exchange in the QROM. In: Guo, J., Steinfeld, R. (eds.) ASIACRYPT 2023, Part IV. LNCS, vol. 14441, pp. 401–433. Springer, Singapore (Dec 2023) [1](#)
37. Pan, J., Zeng, R.: A generic construction of tightly secure password-based authenticated key exchange. In: Guo, J., Steinfeld, R. (eds.) ASIACRYPT 2023, Part VIII. LNCS, vol. 14445, pp. 143–175. Springer, Singapore (Dec 2023) [1](#), [2](#)
38. Sako, K.: An auction protocol which hides bids of losers. In: Imai, H., Zheng, Y. (eds.) PKC 2000. LNCS, vol. 1751, pp. 422–432. Springer, Berlin, Heidelberg (Jan 2000) [2](#)
39. Schwabe, P., Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Seiler, G., Stehlé, D.: CRYSTALS-KYBER. Tech. rep., National Institute of Standards and Technology (2020), available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions> [1](#), [2](#)
40. Unruh, D.: Revocable quantum timed-release encryption. In: Nguyen, P.Q., Oswald, E. (eds.) EUROCRYPT 2014. LNCS, vol. 8441, pp. 129–146. Springer, Berlin, Heidelberg (May 2014) [4](#)
41. Xagawa, K.: Anonymity of NIST PQC round 3 KEMs. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part III. LNCS, vol. 13277, pp. 551–581. Springer, Cham (May / Jun 2022) [4](#)

A Concrete Instantiations

A.1 ElGamal KEM and its Useful Properties

The ElGamal KEM within X-Wing is implemented with a nominal group, and we recall its definition here.

Definition 8 (Nominal Group [3,5]). A nominal group $\mathcal{N} = (\mathcal{G}, g, p, \epsilon_h, \epsilon_u, \exp)$ consists of an efficiently recognizable finite set of elements, $\mathcal{G}$, a base element $g \in \mathcal{G}$, a prime p , a finite set of honest exponents $\epsilon_h \subset \mathbb{Z}$, a finite set of exponents $\epsilon_u \subset \mathbb{Z}/p\mathbb{Z}$, and an efficiently computable exponentiation function $\exp : \mathcal{G} \times \mathbb{Z} \rightarrow \mathcal{G}$, where we write X^y for $\exp(X, y)$. The exponentiation function is required to have the following properties:

1. $(X^y)^z = X^{yz}$ for all $X \in \mathcal{G}$, $y, z \in \mathbb{Z}$
2. The function ϕ defined by $\phi(x) = g^x$ is a bijection from ϵ_u to $\{g^x \mid x \in [1, p-1]\}$.

Let D_H and D_U denote the uniform distribution over ϵ_h and ϵ_u , respectively. Then the statistical distance between them is defined as

$$\Delta_{\mathcal{N}} = \Delta[D_H, D_U] = \frac{1}{2} \sum_{x \in \mathbb{Z}} \left| \Pr_{D_H}(x) - \Pr_{D_U}(x) \right|.$$

Definition 9 (Strong Diffie-Hellman (StCDH) Assumption [3,5]). For a nominal group $\mathcal{N} = (\mathcal{G}, g, p, \epsilon_h, \epsilon_u, \exp)$, we say the StCDH assumption holds over $\mathcal{N}$ if for any PPT adversary $\mathcal{A}$, the advantage of $\mathcal{A}$ against the Strong Diffie-Hellman problem over $\mathcal{N}$ is negligible, which is defined as

$$\text{Adv}_{\mathcal{N}}^{\text{StCDH}}(\mathcal{A}) = \Pr \left[Z = g^{xy} \mid x, y \xleftarrow{\$} \epsilon_u; Z \leftarrow \mathcal{A}^{DH(\cdot, \cdot)}(g^x, g^y) \right].$$

Here $DH(\cdot, \cdot)$ is a decision oracle that on input $(Y, Z) \in \mathcal{G} \times \mathcal{G}$ outputs $\llbracket Y^x = Z \rrbracket$.

According to the IND-CCA security analysis of X-Wing in [5], the curve X25519 [30] can be modeled as a nominal group $\mathcal{N} = (\mathcal{G}, g, p, \epsilon_h, \epsilon_u, \exp)$ where the StCDH assumption holds and $\Delta_{\mathcal{N}} < 2^{-125}$ [3,5].

Let $\text{par} = \mathcal{N}$, we define the key encapsulation mechanism EG-KEM = (KeyGen, Encap, Decap) in Figure 21.

Algorithm KeyGen(par)	Algorithm Encap(pk)	Algorithm Decap(sk, c)
01 $x \xleftarrow{\$} \epsilon_h$	05 $y \xleftarrow{\$} \epsilon_h$	09 $k = \exp(c, \text{sk})$
02 $\text{sk} := x$	06 $k = \exp(\text{pk}, y)$	10 return k
03 $\text{pk} := \exp(g, x)$	07 $c = \exp(g, y)$	
04 return (pk, sk)	08 return (k, c)	

Fig. 21. ElGamal KEM

Lemma 4 (Anonymity of ElGamal KEM). *The EG-KEM KEM is wANO-CCA* secure for any (unbounded) adversary $\mathcal{A}$.*

PROOF. Since the ciphertext $c = g^y$ is generated separately from the public key, the only value involving the information about the challenge bit b is the corresponding challenge key $k^* = g^{\text{sk}_b}$. As long as the adversary does not get k^* , which is the scenario in the wANO-CCA* game, $\mathcal{A}$ can only win the game with probability $1/2$ (even if he has the secret keys). $\square$

A.2 ML-KEM and its Useful Properties

The key encapsulation mechanism ML-KEM-768 [34] can be viewed as a result of a variant of the Fujisaki-Okamoto transformation on the public-key scheme PKE = (Gen, Enc, Dec). Let $\mathbb{B}$ denote the set $\{0, 1, \dots, 255\}$ of unsigned 8-bit integers(bytes). The public key $\text{pk} \in \mathbb{B}^{384k+32}$, the secret key $\text{sk} \in \mathbb{B}^{768k+96}$, and the ciphertext $c \in \mathbb{B}^{32(d_u k + d_v)}$, where $k = 3, d_u = 10, d_v = 4$. Given the system parameters par , the ML-KEM-768 is defined as a tuple of algorithms (KeyGen, Encap, Decap) as shown in Figure 22.

Algorithm KeyGen(par)	Algorithm Encap(pk)	Algorithm Decap(sk, c)
01 $d \xleftarrow{\$} \mathbb{B}^{32}$	09 $m \xleftarrow{\$} \mathbb{B}^{32}$	15 $\text{sk}_{\text{PKE}} := \text{sk}[0 : 384k]$
02 $z \xleftarrow{\$} \mathbb{B}^{32}$	10 if $m = \text{NULL}$	16 $\text{pk}_{\text{PKE}} := \text{sk}[384k : 768k + 32]$
03 if $d = \text{NULL}$ or $z = \text{NULL}$	11 return $\perp$	17 $h := \text{sk}[768k + 32 : 768k + 64]$
04 return $\perp$	12 $(K, r) := G(m \ H(\text{pk}))$	18 $z := \text{sk}[768k + 64 : 768k + 96]$
05 $(\text{pk}_{\text{PKE}}, \text{sk}_{\text{PKE}}) \leftarrow \text{Gen}(d)$	13 $c := \text{Enc}(\text{pk}, m; r)$	19 $m' := \text{Dec}(\text{sk}_{\text{PKE}}, c)$
06 $\text{pk} := \text{pk}_{\text{PKE}}$	14 return (K, c)	20 $(K', r') := G(m' \ h)$
07 $\text{sk}_2 := (\text{sk}_{\text{PKE}} \ \text{pk} \ H(\text{pk}) \ z)$		21 $\tilde{K} := J(z \ c)$
08 return (pk, sk)		22 $c' := \text{Enc}(\text{pk}_{\text{PKE}}, m'; r')$
		23 if $c \neq c'$
		24 $K' := \tilde{K}$
		25 return K'

Fig. 22. ML-KEM-768 KEM [34]

Here we give the properties of ML-KEM-768 that are used in the latter proofs. As shown in Theorem 4, we first prove the wANO-CCA* security of ML-KEM-768. Different than the wANO-CCA security, this notion allows the adversary to query the challenge ciphertext c^* to the decapsulation oracle, and thus implies the wANO-CCA and wANO-PCA security of ML-KEM-768 used in our proofs. When J is a secure PRF and the underlying public-key encryption scheme PKE is δ -secure and γ -spread, we can reduce the wANO-CCA* security of ML-KEM-768 to the wANO-CPA and OW-CPA security of the underlying PKE, and the collision-resistance of $F(\text{pk})$ of PKE in the random oracle model.⁶

⁶ If PKE is WCFR-CPA secure, then we have $\text{Dec}(\text{sk}_{1-b}, \text{Enc}(\text{pk}_b, m)) \neq m$. This means that $\text{Enc}(\text{pk}_{1-b}, m^*) \neq c^*$ and we no longer require γ -spreadness in the proof for Theorem 4.

Then in Theorem 6, we prove the WCPR-CCA security of ML-KEM-768, used in the proof of the ANO-CCA of HKEM in Theorem 1. Roughly speaking, this property means that the decapsulation value of the challenge ciphertext using another secret key should be indistinguishable from a random value. When J is a secure PRF, we can reduce the WCPR-CCA security of ML-KEM-768 to the collision-resistance of $F(\text{pk})$ and the γ -spreadness of the underlying public-key scheme PKE in the random oracle model.⁷

Theorem 4 (wANO-CCA* of ML-KEM-768). *Let $\text{KEM} = (\text{KeyGen}, \text{Encap}, \text{Decap})$ be the key encapsulation mechanism ML-KEM-768, where $\text{PKE} = (\text{Gen}, \text{Enc}, \text{Dec})$ is a δ -correct and γ -spread public-key encryption scheme. Let $\mathcal{A}$ be an adversary against the wANO-CCA* security of ML-KEM-768, making q_G queries to the random oracle $G : \mathcal{M} \times \mathcal{F} \rightarrow \mathcal{K} \times \mathcal{R}$. Then there exists an OW-CPA adversary $\mathcal{B}_1$ and a wANO-CPA adversary $\mathcal{B}_2$ against PKE, and a PRF adversary $\mathcal{C}$ against J such that*

$$\begin{aligned} \text{Adv}_{\text{ML-KEM-768}}^{\text{wANO-CCA}^*}(\mathcal{A}) &\leq \text{Adv}_{\text{PKE}}^{\text{wANO-CPA}}(\mathcal{B}_2) + \text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{B}_1) + \text{Adv}_J^{\text{PRF}}(\mathcal{C}) \\ &\quad + \text{Adv}_{F, \text{PKE}}^{\text{COLL}} + \delta + 2^{-\gamma}, \end{aligned} \quad (11)$$

and

$$\begin{aligned} \text{Time}^*(\mathcal{B}_1) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{B}_1) &\approx \text{Mem}(\mathcal{A}), \\ \text{Time}^*(\mathcal{B}_2) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{B}_2) &\approx \text{Mem}(\mathcal{A}). \end{aligned}$$

PROOF. Let $\mathcal{A}$ be an adversary against the wANO-CCA* security of ML-KEM-768. Consider the sequence of games $\mathbf{G}_0$ - $\mathbf{G}_5$ shown in Figure 23.

Game $\mathbf{G}_0$: This is the wANO-CCA* security game for ML-KEM-768.

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] = \text{Adv}_{\text{ML-KEM-768}}^{\text{wANO-CCA}^*}(\mathcal{A}) + \frac{1}{2}.$$

Game $\mathbf{G}_1$: In this game, we modify the decapsulation oracles $\text{DECAPO}^*(\text{pk}_0, c)$ and $\text{DECAPO}^*(\text{pk}_1, c)$ by replacing the value of $J(z_0, c)$ and $J(z_1, c)$ with random function values $J_0(c)$ and $J_1(c)$ respectively. Since J is a secure PRF, we have

$$\left| \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] \right| \leq \text{Adv}_J^{\text{PRF}}(\mathcal{C}).$$

Game $\mathbf{G}_2$: In this game, we separate the value of K and r by dividing the value of $G(m, f)$ into three cases: $f = F(\text{pk}_0)$, $f = F(\text{pk}_1) \wedge f \neq F(\text{pk}_0)$, and all the rest possibilities form the third case. For the first two cases, we define the output of $G(m, f)$ as $(K = G_{0k}(m), r = G_{0r}(m))$ and $(K = G_{1k}(m), r = G_{1r}(m))$ respectively, where $G_{0k}, G_{0r}, G_{1k}, G_{1r}$ are all random functions. For the last case, we remain the function value as in $\mathbf{G}_1$.

⁷ Similarly, if PKE is WCFR-CCA secure, then we have $m^* \neq m'$, which leads to $\text{Enc}(\text{pk}_{1-b}, m'; r_{1-b}) \neq c^*$ in the second case of the proof for Theorem 6. Therefore, we can get the WCPR-CCA security of ML-KEM-768 trivially without requiring the γ -spreadness and the collision-resistance of $F(\text{pk})$.

Game G_0-G_5		Oracle $\text{DECAPO}^*(pk_0, c)$
01 $(sk_0, pk_0) \leftarrow \text{KeyGen}(\text{par})$		33 parse sk_0 as $(sk_{PKE_0}, pk_0, f_0, z_0)$ $\parallel G_0-G_3$
02 $(sk_1, pk_1) \leftarrow \text{KeyGen}(\text{par})$		34 $m := \text{Dec}(sk_{PKE_0}, c)$ $\parallel G_0-G_3$
03 $b \xleftarrow{\$} \{0, 1\}$		35 $(K, r) := G(m, f_0)$ $\parallel G_0-G_2$
04 $m^* \xleftarrow{\$} \mathcal{M}$		36 $K := G_{0k}(m)$ $\parallel G_3$
05 $G_{0r}, G_{1r} \xleftarrow{\$} \Omega_{G_r}$ $\parallel G_2-G_5$		37 $r := G_{0r}(m)$ $\parallel G_3$
06 $G_{0k}, G_{1k} \xleftarrow{\$} \Omega_{G_k}$ $\parallel G_2-G_3$		38 if $\text{Enc}(pk_0, m; r) = c$ $\parallel G_0-G_3$
07 $G_0, G_1 \xleftarrow{\$} \Omega_{G_{k'}}$ $\parallel G_4-G_5$		39 return K $\parallel G_0-G_3$
08 $(K^*, r^*) := G(m^*, F(pk_b))$ $\parallel G_0-G_2$		40 else return $J(z_0, c)$ $\parallel G_0$
09 $K^* := G_{bk}(m^*)$ $\parallel G_3-G_5$		41 else return $J_0(c)$ $\parallel G_1-G_3$
10 $r^* := G_{br}(m^*)$ $\parallel G_3-G_4$		42 return $G_0(c)$ $\parallel G_4-G_5$
11 $K^* := G_b(c^*)$ $\parallel G_4-G_5$		
12 $r^* \xleftarrow{\$} \mathcal{R}$ $\parallel G_5$		Oracle $\text{DECAPO}^*(pk_1, c)$
13 $c^* := \text{Enc}(pk_b, m^*; r^*)$		43 parse sk_1 as $(sk_{PKE_1}, pk_1, f_1, z_1)$ $\parallel G_0-G_3$
14 $b' \leftarrow \mathcal{A}^{G(\cdot, \cdot), \text{DECAPO}^*(\cdot, \cdot)}(pk_0, pk_1, c^*)$		44 $m := \text{Dec}(sk_{PKE_1}, c)$ $\parallel G_0-G_3$
15 return $\llbracket b = b' \rrbracket$		45 $(K, r) := G(m, f_1)$ $\parallel G_0-G_2$
Oracle $G(m, f)$		46 $K := G_{1k}(m)$ $\parallel G_3$
16 if $f = F(pk_0)$ $\parallel G_2-G_5$		47 $r := G_{1r}(m)$ $\parallel G_3$
17 $K := G_{0k}(m)$ $\parallel G_2-G_3$		48 if $\text{Enc}(pk_1, m; r) = c$ $\parallel G_0-G_3$
18 $r := G_{0r}(m)$ $\parallel G_2-G_5$		49 return K $\parallel G_0-G_3$
19 $c := \text{Enc}(pk_0, m; r)$ $\parallel G_4-G_5$		50 else return $J(z_1, c)$ $\parallel G_0$
20 if $c = c^* \wedge b = 0$ $\parallel G_5$		51 else return $J_1(c)$ $\parallel G_1-G_3$
21 BAD ₁ := true; abort $\parallel G_5$		52 return $G_1(c)$ $\parallel G_4-G_5$
22 $K := G_0(c)$ $\parallel G_4-G_5$		
23 else if $f = F(pk_1)$ $\parallel G_2-G_5$		
24 $K := G_{1k}(m)$ $\parallel G_2-G_3$		
25 $r := G_{1r}(m)$ $\parallel G_2-G_5$		
26 $c := \text{Enc}(pk_1, m; r)$ $\parallel G_4-G_5$		
27 if $c = c^* \wedge b = 1$ $\parallel G_5$		
28 BAD ₁ := true; abort $\parallel G_5$		
29 $K := G_1(c)$ $\parallel G_4-G_5$		
30 else		
31 $(K, r) := G(m, f)$		
32 return (K, r)		

Fig. 23. Games G_0-G_5 for the proof of Theorem 4.

Since this is a conceptual change, we have

$$\Pr[G_2^{\mathcal{A}} \Rightarrow 1] = \Pr[G_1^{\mathcal{A}} \Rightarrow 1].$$

Game G_3 : In this game, instead of computing $(K^*, r^*) := G(m^*, F(pk_b))$, we directly use $G_{bk}(m^*)$ and $G_{br}(m^*)$ to generate K^* and r^* respectively. We do the same modification to the decapsulation oracles. That is, for each bit $\hat{b} \in \{0, 1\}$, we compute K and r by $G_{\hat{b}k}(m)$ and $G_{\hat{b}r}(m)$ respectively in the decapsulation oracle $\text{DECAPO}^*(pk_{\hat{b}}, c)$, where $m := \text{Dec}(sk_{PKE_{\hat{b}}}, c)$.

Since $\mathcal{A}$'s view will only change when $F(pk_0) = F(pk_1)$, we can bound the difference of G_2 and G_3 by the probability of this collision. Thus

$$\left| \Pr[G_3^{\mathcal{A}} \Rightarrow 1] - \Pr[G_2^{\mathcal{A}} \Rightarrow 1] \right| \leq \text{Adv}_{F, PKE}^{\text{COLL}}.$$

Game G_4 In this game, we remove the use of secret keys in the decapsulation oracles by using the ciphertext to compute K . Specifically speaking, when $\mathcal{A}$

queries (pk_0, c) or (pk_1, c) to the decapsulation oracle, we return with $K = G_0(c)$ or $K = G_1(c)$ respectively, where G_0 and G_1 are random functions. In order to keep the consistency, we reprogram the random oracle G like this: For each input (m, f) , if there exists a bit $\hat{b}$ such that $f = F(\text{pk}_{\hat{b}})$, then we compute $r = G_{\hat{b}r}(m)$ and the corresponding ciphertext $c = \text{Enc}(\text{pk}_{\hat{b}}, m; r)$. Then we assign $G_{\hat{b}}(c)$ to the value of the corresponding session key K . In the meantime, instead of getting K^* by $G_{bk}(m^*)$, we compute it by $K^* = G_b(c^*)$.

When PKE is correct, there is a bijection between every m and $c = \text{Enc}(\text{pk}_{\hat{b}}, m; G_{\hat{b}r}(m))$, and thus the modification in $\mathbf{G}_4$ does not change the distribution. Therefore, we can bound the difference of $\mathbf{G}_3$ and $\mathbf{G}_4$ by the δ -correctness of PKE and obtain

$$\left| \Pr[\mathbf{G}_4^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_3^{\mathcal{A}} \Rightarrow 1] \right| \leq \delta.$$

Game $\mathbf{G}_5$ The only change from $\mathbf{G}_4$ is that we choose r uniformly random in this game. As long as $\mathcal{A}$ does not query the random oracle G with $(m^*, F(\text{pk}_b))$, the adversary's view in this game is the same as in $\mathbf{G}_4$. Hence, as shown in Figure 24, we can bound the difference of $\mathbf{G}_4$ and $\mathbf{G}_5$ by the OW-CPA advantage of PKE.

<u>Reduction $\mathcal{B}_1(\text{pk}, c^*)$</u>	<u>Oracle $G(m, f)$</u>
01 $b \xleftarrow{\$} \{0, 1\}$	10 if $f = F(\text{pk}_0)$
02 $\text{pk}_b := \text{pk}$	11 $r := G_{0r}(m)$
03 $(\text{sk}_{1-b}, \text{pk}_{1-b}) \leftarrow \text{KeyGen}(\text{par})$	12 $c := \text{Enc}(\text{pk}_0, m; r)$
04 $G_{0r}, G_{1r} \xleftarrow{\$} \Omega_{G_r}$	13 if $c = c^* \wedge b = 0$
05 $G_0, G_1 \xleftarrow{\$} \Omega_{G_{k'}}$	14 $m^* := m$; BAD ₁ := true
06 $b' \leftarrow \mathcal{A}^{G(\cdot, \cdot), \text{DECAPO}^*(\cdot, \cdot)}(\text{pk}_0, \text{pk}_1, c^*)$	15 $K := G_0(c)$
07 return m^*	16 else if $f = F(\text{pk}_1)$
<u>Oracle $\text{DECAPO}^*(\text{pk}_0, c)$</u>	17 $r := G_{1r}(m)$
08 return $G_0(c)$	18 $c := \text{Enc}(\text{pk}_1, m; r)$
<u>Oracle $\text{DECAPO}^*(\text{pk}_1, c)$</u>	19 if $c = c^* \wedge b = 1$
09 return $G_1(c)$	20 $m^* := m$; BAD ₁ := true
	21 $K := G_1(c)$
	22 else
	23 $(K, r) := G(m, f)$
	24 return (K, r)

Fig. 24. The adversary $\mathcal{B}_1(\text{pk}, c^*)$ against the OW-CPA security of the underlying public-key scheme PKE for the proof of Theorem 4.

In the reduction, after receiving (pk, c^*) , $\mathcal{B}_1$ generates another key pair and chooses a random bit b . For $\mathcal{A}$'s decapsulation query c , $\mathcal{B}_1$ simulates the decapsulation oracle $\text{DECAPO}^*(\text{pk}_0, \cdot)$ ($\text{DECAPO}^*(\text{pk}_1, \cdot)$, resp.) by returning $G_0(c)$ ($G_1(c)$, resp.). For each random oracle query (m, f) , if there exists a bit $\hat{b}$ such that $f = F(\text{pk}_{\hat{b}})$, then $\mathcal{B}_1$ computes the corresponding ciphertext $c = \text{Enc}(\text{pk}_{\hat{b}}, m; G_{\hat{b}r}(m))$ and checks if $c = c^*$. We define the event **BAD**₁ occurs

when $c = c^*$ and $\hat{b} = b$, and if BAD_1 occurs, $\mathcal{B}_1$ outputs the corresponding m as the guess of the plaintext of c^* . Hence, we have

$$\left| \Pr[\mathbf{G}_5^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_4^{\mathcal{A}} \Rightarrow 1] \right| \leq \Pr[\text{BAD}_1] \leq \text{Adv}_{\text{PKE}}^{\text{OW-CPA}}(\mathcal{B}_1).$$

Reduction $\mathcal{B}_2(\text{pk}_0, \text{pk}_1, c^*)$	Oracle $G(m, f)$
01 $G_{0r}, G_{1r} \xleftarrow{\$} \Omega_{G_r}$	07 if $f = F(\text{pk}_0)$
02 $G_0, G_1 \xleftarrow{\$} \Omega_{G_{k'}}$	08 $r := G_{0r}(m)$
03 $b' \leftarrow \mathcal{A}^{G(\cdot, \cdot), \text{DECAPO}^*(\cdot, \cdot)}(\text{pk}_0, \text{pk}_1, c^*)$	09 $c := \text{Enc}(\text{pk}_0, m; r)$
04 return b'	10 if $c = c^*$
	11 BAD := true ; abort
<u>Oracle $\text{DECAPO}^*(\text{pk}_0, c)$</u>	12 $K := G_0(c)$
05 return $G_0(c)$	13 else if $f = F(\text{pk}_1)$
<u>Oracle $\text{DECAPO}^*(\text{pk}_1, c)$</u>	14 $r := G_{1r}(m)$
06 return $G_1(c)$	15 $c := \text{Enc}(\text{pk}_1, m; r)$
	16 if $c = c^*$
	17 BAD := true ; abort
	18 $K := G_1(c)$
	19 else
	20 $(K, r) := G(m, f)$
	21 return (K, r)

Fig. 25. The adversary $\mathcal{B}_2(\text{pk}_0, \text{pk}_1, c^*)$ against the wANO-CPA security of the underlying public-key scheme PKE for the proof of Theorem 4.

Finally, we can reduce the success probability of $\mathcal{A}$ in $\mathbf{G}_5$ to the wANO-CPA security of PKE as described in Figure 25. In the reduction, since $\mathcal{B}_2$ does not know the bit b , the algorithm aborts when the re-encryption of m equals to c^* (without justifying whether the public key corresponds to the bit b) in the simulation of G . We define this event as **BAD** and obviously if BAD_1 occurs, then **BAD** must happen. There is only one case when **BAD** happens but BAD_1 does not: $\mathcal{A}$ queries the random oracle G with $(m, F(\text{pk}_{1-b}))$ such that $\text{Enc}(\text{pk}_{1-b}, m; G_{1-b,r}(m)) = c^*$. Since $G_{1-b,r}$ is a random function, we can bound the probability of this case by the γ -spreadness of PKE and obtain

$$\Pr[\mathbf{G}_5^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_{\text{PKE}}^{\text{wANO-CPA}}(\mathcal{B}_2) + 2^{-\gamma} + \frac{1}{2}.$$

By combining all the bounds above, we get Equation (11). $\square$

Theorem 5 (C2PRI of ML-KEM-768 [5]). *Let G and J be independent random oracles with output size 512 and 256 bits. Let $\mathcal{A}$ be an adversary against the C2PRI security of ML-KEM-768, making at most q_G and q_J queries to the*

random oracles G and J respectively. Then the C2PRI advantage of adversary $\mathcal{A}$ against ML-KEM-768 is

$$\text{Adv}_{\text{ML-KEM-768}}^{\text{C2PRI}}(\mathcal{A}) \leq \frac{q_G + q_J + 2}{2^{256}}. \quad (12)$$

Theorem 6 (WCPR-CCA of ML-KEM-768). *Let $\text{KEM} = (\text{KeyGen}, \text{Encap}, \text{Decap})$ be the key encapsulation mechanism ML-KEM-768, where $\text{PKE} = (\text{Gen}, \text{Enc}, \text{Dec})$ is a γ -spread public-key encryption scheme. Let $\mathcal{A}$ be an adversary against the WCPR-CCA security of ML-KEM-768, making q_G queries to the random oracle $G : \mathcal{M} \times \mathcal{F} \rightarrow \mathcal{K} \times \mathcal{R}$. Then there exists a PRF adversary $\mathcal{B}$ against J such that*

$$\text{Adv}_{\text{ML-KEM-768}}^{\text{WCPR-CCA}}(\mathcal{A}) \leq \text{Adv}_{F, \text{PKE}}^{\text{COLL}} + \text{Adv}_J^{\text{PRF}}(\mathcal{B}) + 2^{-\gamma}.$$

PROOF. For any $c^* = \text{Enc}(\text{pk}_b, m^*; r_b)$, let $m' = \text{Dec}(\text{sk}_{1-b}, c^*)$.

If $\text{Enc}(\text{pk}_{1-b}, m^*; r_{1-b}) \neq c^*$, which means that the re-encryption check failed, then $K_0 = J(z_{1-b}, c^*)$ is indistinguishable from a random value when J is a secure PRF.

If $\text{Enc}(\text{pk}_{1-b}, m^*; r_{1-b}) = c^*$, then $K_0 = K'$ where $(K', r_{1-b}) := G(m', F(\text{pk}_{1-b}))$. In this case, if $F(\text{pk}_0) \neq F(\text{pk}_1)$, then r' is uniformly random because G is modeled as a random oracle. Since PKE is γ -spread, we have

$$\Pr[\text{Enc}(\text{pk}_{1-b}, m'; r_{1-b}) = c^*] \leq 2^{-\gamma} + \text{Adv}_{F, \text{PKE}}^{\text{COLL}}.$$

□

Game $\text{WCPR-CCA}_{\text{ML-KEM-768}}^{\mathcal{A}}$	Oracle $\text{DECAPO}^*(\text{pk}_0, c)$
01 $(\text{sk}_0, \text{pk}_0) \leftarrow \text{KeyGen}(\text{par})$	18 if $c = c^*$
02 $(\text{sk}_1, \text{pk}_1) \leftarrow \text{KeyGen}(\text{par})$	19 return $\perp$
03 $b \xleftarrow{\$} \{0, 1\}$	20 parse sk_0 as $(\text{sk}_{\text{PKE}_0}, \text{pk}_0, f_0, z_0)$
04 $m^* \xleftarrow{\$} \mathcal{M}$	21 $m := \text{Dec}(\text{sk}_{\text{PKE}_0}, c)$
05 $(K, r^*) := G(m^*, F(\text{pk}_b))$	22 $(K, r) := G(m, f_0)$
06 $c^* := \text{Enc}(\text{pk}_b, m^*; r^*)$	23 if $\text{Enc}(\text{pk}_0, m; r) = c$
07 parse sk_{1-b} as $(\text{sk}_{\text{PKE}_{1-b}}, \text{pk}_{1-b}, f_{1-b}, z_{1-b})$	24 return K
08 $m' := \text{Dec}(\text{sk}_{\text{PKE}_{1-b}}, c^*)$	25 else return $J(z_0, c)$
09 $(K', r') := G(m', f_{1-b})$	Oracle $\text{DECAPO}^*(\text{pk}_1, c)$
10 if $c^* = \text{Enc}(\text{pk}_{1-b}, m'; r')$	26 if $c = c^*$
11 $K_0 := K'$	27 return $\perp$
12 else	28 parse sk_1 as $(\text{sk}_{\text{PKE}_1}, \text{pk}_1, f_1, z_1)$
13 $K_0 := J(z_{1-b}, c)$	29 $m := \text{Dec}(\text{sk}_{\text{PKE}_1}, c)$
14 $K_1 \xleftarrow{\$} \mathcal{K}$	30 $(K, r) := G(m, f_1)$
15 $b' \xleftarrow{\$} \{0, 1\}$	31 if $\text{Enc}(\text{pk}_1, m; r) = c$
16 $\hat{b} \leftarrow \mathcal{A}^{G(\cdot, \cdot), \text{DECAPO}(\cdot, \cdot)}(\text{pk}_0, \text{pk}_1, b, c^*, K_{b'})$	32 return K
17 return $[\hat{b} = b']$	33 else return $J(z_1, c)$

Fig. 26. Definition for the WCPR-CCA security of ML-KEM-768.

A.3 Construction of X-Wing

The key encapsulation mechanism X-Wing [5] can be modeled as a hybrid KEM using the generic construction in Section 4.1 to instantiate KEM_1 and KEM_2 with ML-KEM-768 and EG-KEM respectively. Given the system parameters $\text{par} := (\text{par}_1, \mathcal{N})$, where $\mathcal{N} = (\mathcal{G}, g, p, \epsilon_h, \epsilon_u, \text{exp})$ is the nominal group, the ML-KEM-768 KEM, and the SHA3-256 hash function, the X-Wing KEM is defined as a tuple of algorithms (KeyGen, Encap, Decap) as shown in Figure 27.

Algorithm KeyGen(par)	Algorithm Encap(pk)	Algorithm Decap(sk, c)
01 $(\text{sk}_1, \text{pk}_1) \leftarrow \text{ML-KEM-768.KeyGen}(\text{par}_1)$	07 parse pk as $(\text{pk}_1, \text{pk}_2)$	16 parse sk as $(\text{sk}_1, \text{sk}_2, \text{pk}_2)$
02 $\text{sk}_2 \xleftarrow{\$} \mathbb{B}^{32}$	08 $\text{sk}_e \xleftarrow{\$} \mathbb{B}^{32}$	17 parse c as (c_1, c_2)
03 $\text{pk}_2 := \text{exp}(g, \text{sk}_2)$	09 $c_2 := \text{exp}(g, \text{sk}_e)$	18 $k_1 := \text{ML-KEM-768.Decap}(\text{sk}_1, c_1)$
04 $\text{sk} := (\text{sk}_1, \text{sk}_2, \text{pk}_2)$	10 $(k_1, c_1) \leftarrow \text{ML-KEM-768.Encap}(\text{pk}_1)$	19 $k_2 := \text{exp}(c_2, \text{sk}_2)$
05 $\text{pk} := (\text{pk}_1, \text{pk}_2)$	11 $k_2 := \text{exp}(\text{pk}_2, \text{sk}_e)$	20 $s := "\backslash.\text{}/\text{~}\backslash" \ k_1 \ k_2 \ c_2 \ \text{pk}_2$
06 return (pk, sk)	12 $s := \text{SHA3-256}(s)$	21 $k := \text{SHA3-256}(s)$
	13 $k := \text{SHA3-256}(s)$	22 return k
	14 $c := (c_1, c_2)$	
	15 return (k, c)	

Fig. 27. X-Wing Hybrid KEM [5]

By implementing KEM_1 with ML-KEM-768 and KEM_2 with EG-KEM, and modeling H as the random oracle, we can apply our results about the anonymity of the generic construction HKEM in Section 4.2 to X-Wing. By using the properties of ML-KEM-768 and EG-KEM proved in Supporting Material A.1 and A.2, we get the anonymity of X-Wing and conclude them in the following corollaries.

If ML-KEM-768 is IND-CCA secure, then we can apply Theorem 1 to X-Wing and get its anonymity. As stated in Lemma 4, EG-KEM is unconditionally wANO-CCA* secure, and thus wANO-PCA secure. This means that we only need security assumptions for ML-KEM-768 to prove the anonymity of X-Wing under this case.

By combining Theorem 1 and Lemma 4, we get the ANO-CCA security of X-Wing by memory-tight reductions, as described in the following corollary:

Corollary 1 (Anonymity of X-Wing). *Let $\mathcal{A}$ be an adversary against the IND-CCA security of X-Wing, making at most q_H queries to the random oracle H and q_D queries to the decapsulation oracles. Then there exist an IND-CCA adversary $\mathcal{B}_1$, a WCP-CCA adversary $\mathcal{B}_2$, and a wANO-CCA adversary $\mathcal{B}_3$ against ML-KEM-768 such that*

$$\begin{aligned} \text{Adv}_{\text{X-Wing}}^{\text{ANO-CCA}}(\mathcal{A}) &\leq \text{Adv}_{\text{ML-KEM-768}}^{\text{IND-CCA}}(\mathcal{B}_1) + \text{Adv}_{\text{ML-KEM-768}}^{\text{WCP-CCA}}(\mathcal{B}_2) + \text{Adv}_{\text{ML-KEM-768}}^{\text{wANO-CCA}}(\mathcal{B}_3) \\ &\quad + \delta + \frac{1 + 2q_H + 2q_D}{2^{256}}, \end{aligned} \tag{13}$$

and

$$\begin{aligned} \text{Time}(\mathcal{B}_1) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{B}_1) &\approx \text{Mem}(\mathcal{A}), \\ \text{Time}(\mathcal{B}_2) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{B}_2) &\approx \text{Mem}(\mathcal{A}), \\ \text{Time}(\mathcal{B}_3) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{B}_3) &\approx \text{Mem}(\mathcal{A}). \end{aligned}$$

If StCDH assumption holds with the nominal group $\mathcal{N}$, then we can apply the result from [5, Theorem 1], which proves the IND-CCA security of X-Wing under the situation that ML-KEM-768 may not be IND-CCA secure. We adapt the same technique in Section 4.2 (cf. $\mathbf{G}_6$ in the proof for Theorem 1) to give a memory-tight proof of the IND-CCA security of X-Wing. Our probability and running time statements in Theorem 7 are the same as in [5], but our memory consumption is tight.

Theorem 7 (IND-CCA of X-Wing, revisited). *Let $\mathcal{A}$ be an adversary against the IND-CCA security of X-Wing, making at most q_H queries to the random oracle H . Then there exist adversaries $\mathcal{B}$ and $\mathcal{C}$ such that*

$$\text{Adv}_{\text{X-Wing}}^{\text{IND-CCA}}(\mathcal{A}) \leq 2\Delta_{\mathcal{N}} + \text{Adv}_{\text{ML-KEM-768}}^{\text{C2PRI}}(\mathcal{B}) + \text{Adv}_{\mathcal{N}}^{\text{StCDH}}(\mathcal{C}), \quad (14)$$

and

$$\begin{aligned} \text{Time}(\mathcal{B}) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{B}) &\approx \text{Mem}(\mathcal{A}), \\ \text{Time}(\mathcal{C}) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{C}) &\approx \text{Mem}(\mathcal{A}). \end{aligned}$$

PROOF. Let $\mathcal{A}$ be an adversary against the IND-CCA security of X-Wing. Consider the sequence of games $\mathbf{G}_0$ - $\mathbf{G}_4$ shown in Figure 28, where we model ML-KEM-768 as a black box denoted as KEM_1 . Without loss of generality, here we assume the input of H has the form of $(\text{label} \| k_1 \| k_2 \| c_2 \| \text{pk}_2)$, where pk_2 is defined as in Line 05 and other values are arbitrary.

Game $\mathbf{G}_0$ - $\mathbf{G}_4$	Oracle $\text{DECAPO}(c)$
01 $(\text{sk}_1, \text{pk}_1) \leftarrow \text{KeyGen}(\text{par}_1)$	21 if $c = c^*$: return $\perp$
02 $x \xleftarrow{\$} \epsilon_h$	22 parse c as (c_1, c_2)
03 $x \xleftarrow{\$} \epsilon_u$	23 $k_1 := \text{Decap}(\text{sk}_1, c_1)$
04 $\text{sk}_2 := x$	24 if $c_1 \neq c_1^* \wedge k_1 = k_1^*$
05 $\text{pk}_2 := \text{exp}(g, x)$	25 $\text{BAD}_1 := \text{true}$; abort
06 $\text{pk} := (\text{pk}_1, \text{pk}_2)$	26 $k_2 := \text{exp}(c_2, x)$
07 $(k_1^*, c_1^*) \leftarrow \text{Encap}_1(\text{pk}_1)$	27 $K := H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$
08 $y \xleftarrow{\$} \epsilon_h$	28 $K := H'(\text{label} \ k_1 \ * \ c_2 \ \text{pk}_2)$
09 $y \xleftarrow{\$} \epsilon_u$	29 return K
10 $c_2^* := \text{exp}(g, y)$	Oracle $H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$
11 $k_2^* := \text{exp}(\text{pk}_2, y)$	30 if $k_2 = \text{exp}(c_2, x)$
12 $c^* := (c_1^*, c_2^*)$	31 if $c_2 = c_2^*$
13 $H' \xleftarrow{\$} \Omega_H$	32 $\text{BAD}_2 := \text{true}$; abort
14 $K_0 := H'(\text{label} \ k_1^* \ k_2^* \ c_2^* \ \text{pk}_2)$	33 return $H'(\text{label} \ k_1 \ * \ c_2 \ \text{pk}_2)$
15 $K_0 := H(\text{label} \ k_1^* \ * \ c_2^* \ \text{pk}_2)$	34 return $H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$
16 $K_0 \xleftarrow{\$} \mathcal{K}$	
17 $K_1 \xleftarrow{\$} \mathcal{K}$	
18 $b \xleftarrow{\$} \{0, 1\}$	
19 $b' \leftarrow \mathcal{A}^{H(\cdot), \text{DECAPO}^*(\cdot)}(\text{pk}, K_b, c^*)$	
20 return $\llbracket b = b' \rrbracket$	

Fig. 28. Games $\mathbf{G}_0$ - $\mathbf{G}_4$ for the proof of Theorem 7.

Game $\mathbf{G}_0$: This is the IND-CCA security game for X-Wing.

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] = \text{Adv}_{\text{X-Wing}}^{\text{IND-CCA}}(\mathcal{A}) + \frac{1}{2}.$$

Game $\mathbf{G}_1$: This is exactly the same as the first step in [5, Theorem 1], where we change the distribution from ϵ_h to ϵ_u . This is a preparation step for reducing to the StCDH assumption and induces a statistical difference.

$$\left| \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] \right| \leq 2\Delta_{\mathcal{N}}.$$

Game $\mathbf{G}_2$: In this game, for all the hash values $H(\text{label} \| k_1 \| k_2 \| c_2 \| \text{pk}_2)$ which satisfy $k_2 = \exp(c_2, x) \wedge \text{pk}_2 = \exp(g, x)$ (i.e., $\text{Decap}_2(\text{sk}_2, c_2) = k_2$), we reprogram them by the corresponding random function value $H'(\text{label}, k_1, \star, c_2, \text{pk}_2)$. By doing this, we do not need the secret key $\text{sk}_2 := x$ in the decapsulation oracle. Since there is a bijection between (k_2, c_2, pk_2) and (c_2, pk_2) , this reprogramming does not change the distribution. Therefore,

$$\Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] = \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1].$$

Game $\mathbf{G}_3$: In this game, if $\mathcal{A}$ queries the decapsulation oracle with (c_1, c_2) where $c_1 \neq c_1^*$ has the same decapsulation value with c_1^* , then we denote this event as BAD_1 and aborts the game. Similarly as the proof in [5, Theorem 1], we use the memory-tight reduction shown in Figure 29 to bound the probability of BAD_1 by the C2PRI advantage of KEM_1 :

$$\left| \Pr[\mathbf{G}_3^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] \right| \leq \Pr[\text{BAD}_1] \leq \text{Adv}_{\text{KEM}_1}^{\text{C2PRI}}(\mathcal{B}).$$

Game $\mathbf{G}_4$: In this game, instead of computing K_0 by $H(\text{label} \| k_1^* \| k_2^* \| c_2^* \| \text{pk}_2)$, we choose it uniformly random from $\mathcal{K}$. Since BAD_1 occurs if the output of $\text{DECAPO}(c)$ is $K = H(\text{label} \| k_1^* \| k_2^* \| c_2^* \| \text{pk}_2)$, $\mathcal{A}$ can distinguish the game difference only when querying the random oracle H with $(\text{label} \| k_1^* \| k_2^* \| c_2^* \| \text{pk}_2)$. Let BAD_2 denote the event that $\mathcal{A}$ queries $H(\text{label} \| k_1 \| k_2 \| c_2 \| \text{pk}_2)$ satisfying $k_2 = \exp(c_2, x) = k_2^*$ and $c_2 = c_2^*$.

As shown in Figure 30, we can reduce the probability of BAD_2 to the StCDH assumption. While [5, Theorem 1] uses the table $\mathbf{E}[(c_2, k_1)]$ to keep the consistency of the decapsulation oracle DECAPO^* and the random oracle H , we get rid of this memory loss by the use of the random function H' introduced in the preparation game hop $\mathbf{G}_2$.

Since $\mathcal{A}$'s view is the same in $\mathbf{G}_3$ and $\mathbf{G}_4$ if BAD_2 does not happen, we have

$$\left| \Pr[\mathbf{G}_4^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_3^{\mathcal{A}} \Rightarrow 1] \right| \leq \Pr[\text{BAD}_2] \leq \text{Adv}_{\mathcal{N}}^{\text{StCDH}}(\mathcal{C}).$$

Both K_0 and K_1 are chosen uniformly random in $\mathbf{G}_4$, and thus

$$\Pr[\mathbf{G}_4^{\mathcal{A}} \Rightarrow 1] = \frac{1}{2}.$$

Reduction $\mathcal{B}(\text{sk}_1, \text{pk}_1, k_1^*, c_1^*)$	Oracle $\text{DECAPO}(c)$
01 $x \xleftarrow{\$} \epsilon_u$	15 if $c = c^*$: return $\perp$
02 $\text{sk}_2 := x$	16 parse c as (c_1, c_2)
03 $\text{pk}_2 := \text{exp}(g, x)$	17 $k_1 := \text{Decap}(\text{sk}_1, c_1)$
04 $\text{pk} := (\text{pk}_1, \text{pk}_2)$	18 if $c_1 \neq c_1^* \wedge k_1 = k_1^*$
05 $y \xleftarrow{\$} \epsilon_u$	19 $\text{BAD}_1 := \text{true}$; abort
06 $c_2^* := \text{exp}(g, y)$	20 $K := H'(\text{label} \ k_1 \ \star \ c_2 \ \text{pk}_2)$
07 $k_2^* := \text{exp}(\text{pk}_2, y)$	21 return K
08 $c^* := (c_1^*, c_2^*)$	
09 $H' \xleftarrow{\$} \Omega_H$	Oracle $H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$
10 $K_0 := H'(\text{label} \ k_1^* \ \star \ c_2^* \ \text{pk}_2)$	22 if $k_2 = \text{exp}(c_2, x)$
11 $K_1 \xleftarrow{\$} \mathcal{K}$	23 return $H'(\text{label} \ k_1 \ \star \ c_2 \ \text{pk}_2)$
12 $b \xleftarrow{\$} \{0, 1\}$	24 return $H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$
13 $b' \leftarrow \mathcal{A}^{H(\cdot), \text{DECAPO}^*(\cdot)}(\text{pk}, K_b, c^*)$	
14 return $\llbracket b = b' \rrbracket$	

Fig. 29. The adversary $\mathcal{B}(\text{sk}_1, \text{pk}_1, k_1^*, c_1^*)$ against the C2PRI security of KEM_1 for the proof of Theorem 7.

Reduction $\mathcal{C}^{\text{DH}(\cdot, \cdot)}(X, Y)$	Oracle $\text{DECAPO}(c)$
01 $(\text{sk}_1, \text{pk}_1) \leftarrow \text{KeyGen}(\text{par}_1)$	13 if $c = c^*$: return $\perp$
02 $(k_1^*, c_1^*) \leftarrow \text{Encap}_1(\text{pk}_1)$	14 parse c as (c_1, c_2)
03 $\text{pk}_2 := X$	15 $k_1 := \text{Decap}(\text{sk}_1, c_1)$
04 $\text{pk} := (\text{pk}_1, \text{pk}_2)$	16 if $c_1 \neq c_1^* \wedge k_1 = k_1^*$
05 $c_2^* := Y$	17 $\text{BAD}_1 := \text{true}$; abort
06 $c^* := (c_1^*, c_2^*)$	18 $K := H'(\text{label} \ k_1 \ \star \ c_2 \ \text{pk}_2)$
07 $H' \xleftarrow{\$} \Omega_H$	19 return K
08 $K_0 := H'(\text{label} \ k_1^* \ \star \ c_2^* \ \text{pk}_2)$	Oracle $H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$
09 $K_1 \xleftarrow{\$} \mathcal{K}$	20 if $\text{DH}(c_2, k_2) = 1$
10 $b \xleftarrow{\$} \{0, 1\}$	21 if $c_2 = Y$
11 $b' \leftarrow \mathcal{A}^{H(\cdot), \text{DECAPO}^*(\cdot)}(\text{pk}, K_b, c^*)$	22 $k_2^* := k_2$; $\text{BAD}_2 := \text{true}$; abort
12 return k_2^*	23 return $H'(\text{label} \ k_1 \ \star \ c_2 \ \text{pk}_2)$
	24 return $H(\text{label} \ k_1 \ k_2 \ c_2 \ \text{pk}_2)$

Fig. 30. The adversary $\mathcal{C}^{\text{DH}(\cdot, \cdot)}(X, Y)$ against the StCDH assumption for the proof of Theorem 7. In the reduction, $\mathcal{C}$ queries the gap oracle at most q_H times.

We have Equation (14) by combining all the bounds above.

□

As stated in Lemma 4, EG-KEM is unconditionally wANO-CCA* secure, and thus wANO-PCA secure. By combining this fact with Lemma 3 and Theorem 7,

we can get the ANO-CCA security of X-Wing without requiring the IND-CCA security of ML-KEM-768, as described in the following corollary:

Corollary 2 (Anonymity of X-Wing). *Let $\mathcal{A}$ be an adversary against the ANO-CCA security of X-Wing, making at most q_H queries to the random oracle H and q_D queries to the decapsulation oracles. Then there exist a StCDH adversary $\mathcal{B}$, a C2PRI adversary $\mathcal{C}_1$ and a wANO-PCA adversary $\mathcal{C}_2$ against the ML-KEM-768 such that*

$$\text{Adv}_{\text{X-Wing}}^{\text{ANO-CCA}}(\mathcal{A}) \leq 2\Delta_{\mathcal{N}} + \text{Adv}_{\mathcal{N}}^{\text{StCDH}}(\mathcal{B}) + \text{Adv}_{\text{ML-KEM-768}}^{\text{C2PRI}}(\mathcal{C}_1) + \text{Adv}_{\text{ML-KEM-768}}^{\text{wANO-PCA}}(\mathcal{C}_2), \quad (15)$$

and

$$\begin{aligned} \text{Time}(\mathcal{B}) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{B}) &\approx \text{Mem}(\mathcal{A}), \\ \text{Time}(\mathcal{C}_1) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{C}_1) &\approx \text{Mem}(\mathcal{A}), \\ \text{Time}(\mathcal{C}_2) &\approx \text{Time}(\mathcal{A}), & \text{Mem}^*(\mathcal{C}_2) &= \text{Mem}(\mathcal{A}) + \log |\mathcal{L}_H| + \log |D|. \end{aligned}$$

Here $\mathcal{L}_H$ and D denote the lists used to simulate the random oracle and the decapsulation oracle respectively, where $|\mathcal{L}_H| = \mathcal{O}(q_H)$ and $|D| = \mathcal{O}(q_D)$.

A.4 Instantiation of Our Variant of X-Wing

Let X-Wing' be the hybrid key encapsulation mechanism HKEM' in Section 5 instantiated by ML-KEM-768 as KEM₁ and EG-KEM as KEM₂.

Since the public-key collision advantage of EG-KEM will be negligible if the distribution is ϵ_u , we have $\text{Adv}_{\text{EG-KEM}}^{\text{COLL}} = \frac{1}{p} + 2\Delta_{\mathcal{N}}$. Same as former analysis, we also have the wANO-PCA security of EG-KEM unconditionally according to Lemma 4. This means that we only need the wANO-PCA security of ML-KEM-768 to get the wANO-CCA* security of X-Wing'. Moreover, the IND-CCA security of X-Wing' requires the OW-PCA security of either ML-KEM-768 or EG-KEM, where the latter can be reduced to the StCDH assumption on the nominal group $\mathcal{N}$.

Therefore, if ML-KEM-768 is wANO-PCA and OW-PCA secure, then we can get the anonymity of X-Wing' without any requirements of EG-KEM. Or, if StCDH assumption holds, then we only need the wANO-PCA security of ML-KEM-768 without any other security notion to get the anonymity of X-Wing'.

By combining Lemma 4, Theorem 2, and Lemma 1, we can get the ANO-CCA security of X-Wing' by memory-tight reductions as in the following corollary:

Corollary 3 (Anonymity of X-Wing'). *Let $\mathcal{A}$ be an adversary against the ANO-CCA security of X-Wing', making at most q_H queries to the random oracle H and q_D queries to the decapsulation oracles. Then there exist an OW-PCA adversary $\mathcal{B}_1$ and a wANO-PCA adversary $\mathcal{B}_2$ against the ML-KEM-768, and a StCDH adversary $\mathcal{C}$ against the nominal group $\mathcal{N}$ such that*

$$\begin{aligned} \text{Adv}_{\text{X-Wing}'}^{\text{ANO-CCA}}(\mathcal{A}) &\leq \text{Adv}_{\text{ML-KEM-768}}^{\text{wANO-PCA}}(\mathcal{B}_1) + \frac{1}{p} + 2\Delta_{\mathcal{N}} \\ &\quad + \min(\text{Adv}_{\text{ML-KEM-768}}^{\text{OW-PCA}}(\mathcal{B}_2), \text{Adv}_{\mathcal{N}}^{\text{StCDH}}(\mathcal{C})), \end{aligned} \quad (16)$$

and

$$\begin{aligned}\mathbf{Time}(\mathcal{B}_1) &\approx \mathbf{Time}(\mathcal{A}), & \mathbf{Mem}^*(\mathcal{B}_1) &\approx \mathbf{Mem}(\mathcal{A}), \\ \mathbf{Time}(\mathcal{B}_2) &\approx \mathbf{Time}(\mathcal{A}), & \mathbf{Mem}^*(\mathcal{B}_2) &\approx \mathbf{Mem}(\mathcal{A}), \\ \mathbf{Time}(\mathcal{C}) &\approx \mathbf{Time}(\mathcal{A}), & \mathbf{Mem}^*(\mathcal{C}) &\approx \mathbf{Mem}(\mathcal{A}).\end{aligned}$$